

Sistem Terdistribusi

IF2228

05: Penamaan

September 2026

Sistem Terdistribusi

IF2228

Pengampu

Yoga Dwitya P.

Today...

- Last Session:

- Architectures

- Today's Session:

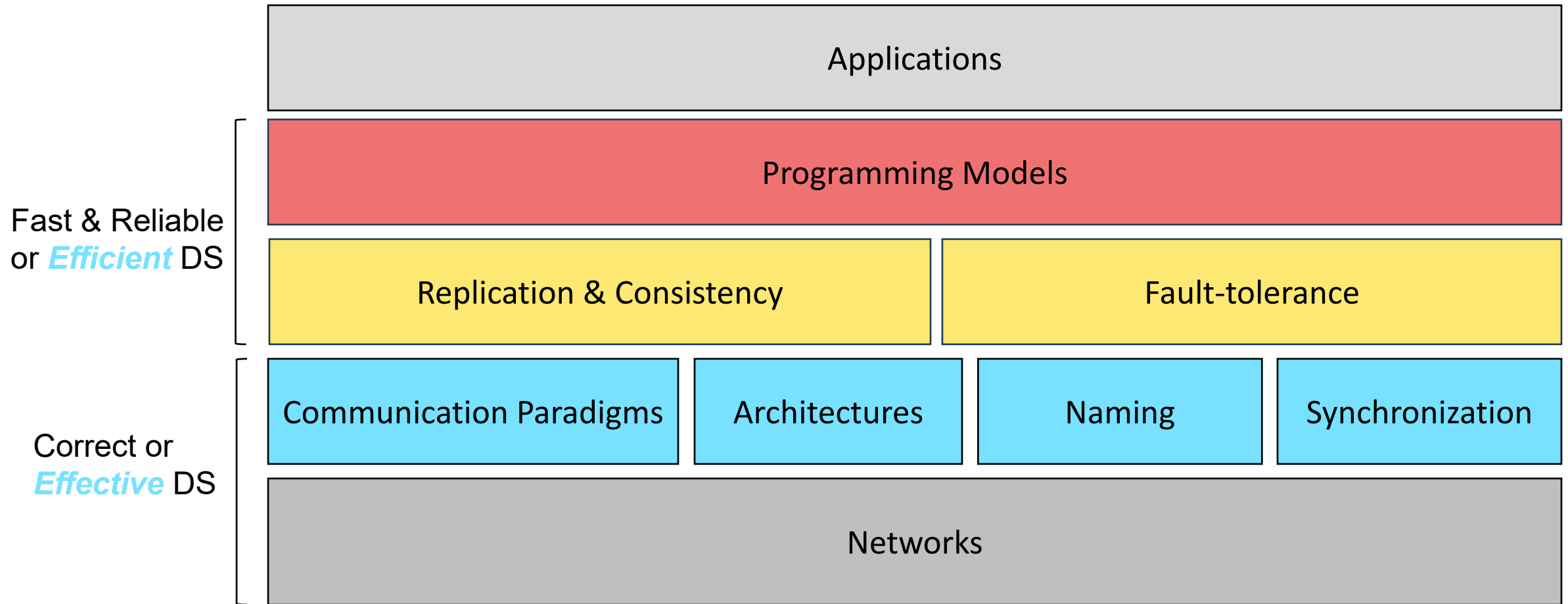
- Naming (*Chapter 5 in Steen's and Tanenbaum's book*)

- Announcements:

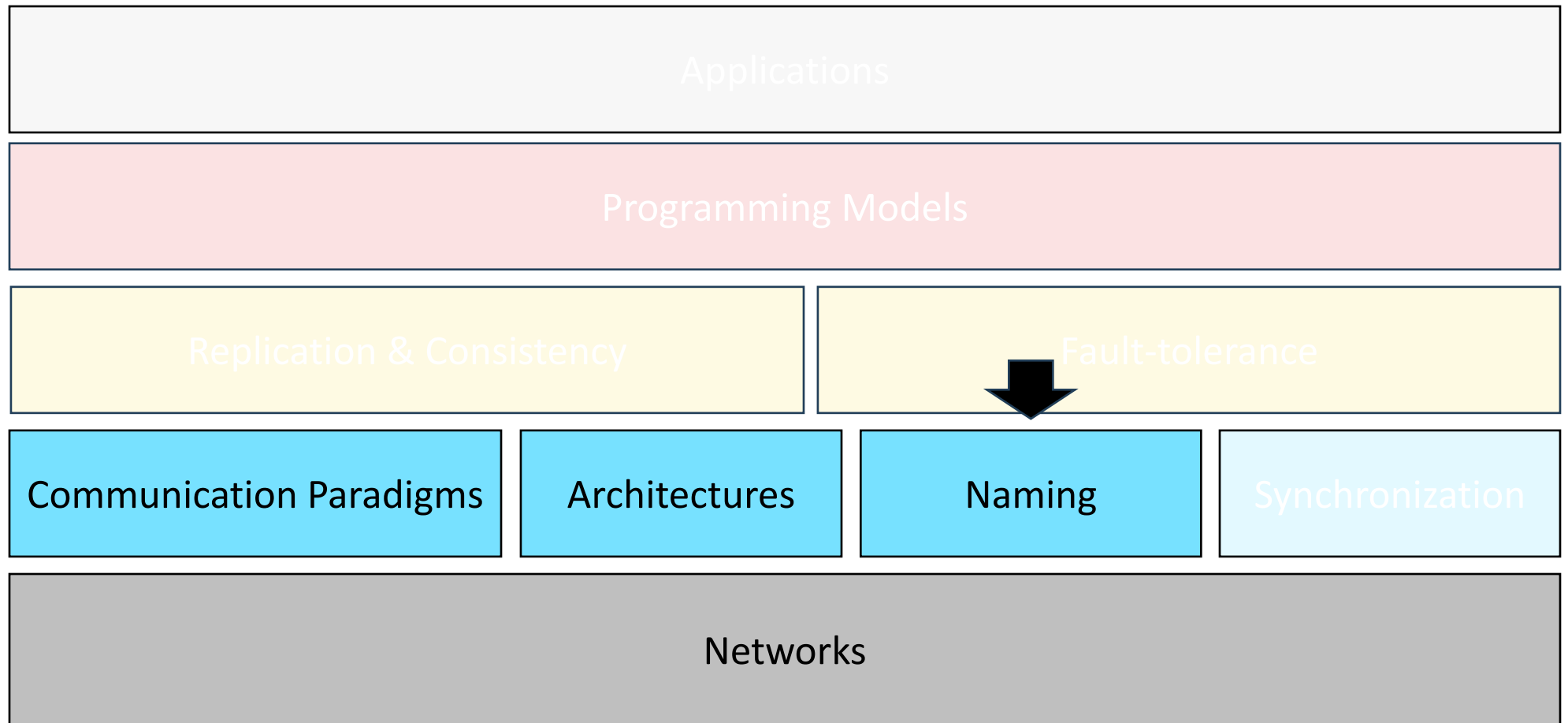
- P1 design report is due today by midnight
- Quiz I is on Sep 29



Course Map

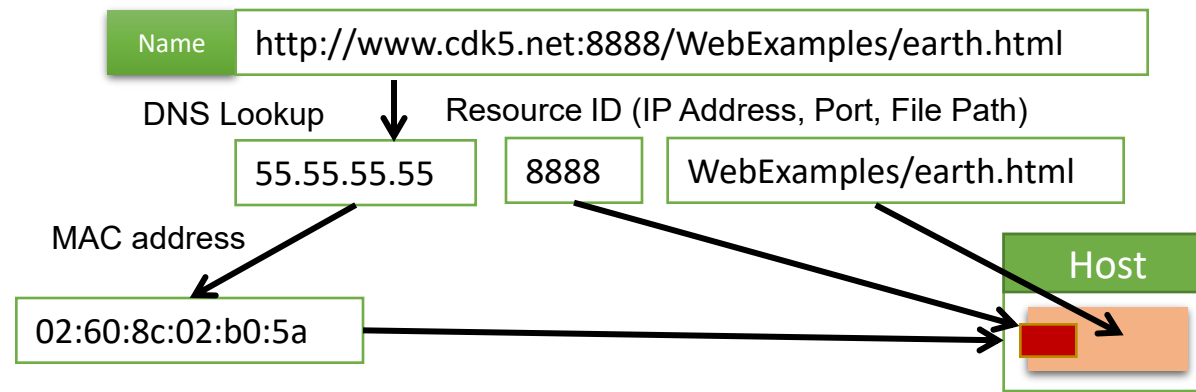


Course Map



Naming

- *Names* are used to uniquely identify *entities* in distributed systems
 - *Entities* may be processes, remote objects, newsgroups, etc.,
- Names are mapped to entities' locations using *name resolution*
- An example of name resolution:



Names, Addresses, and Identifiers

- An *entity* can be identified by three types of references

a) Name

- A name is a set of bits or characters that references an entity
- Names can be human-friendly (or not)

b) Address

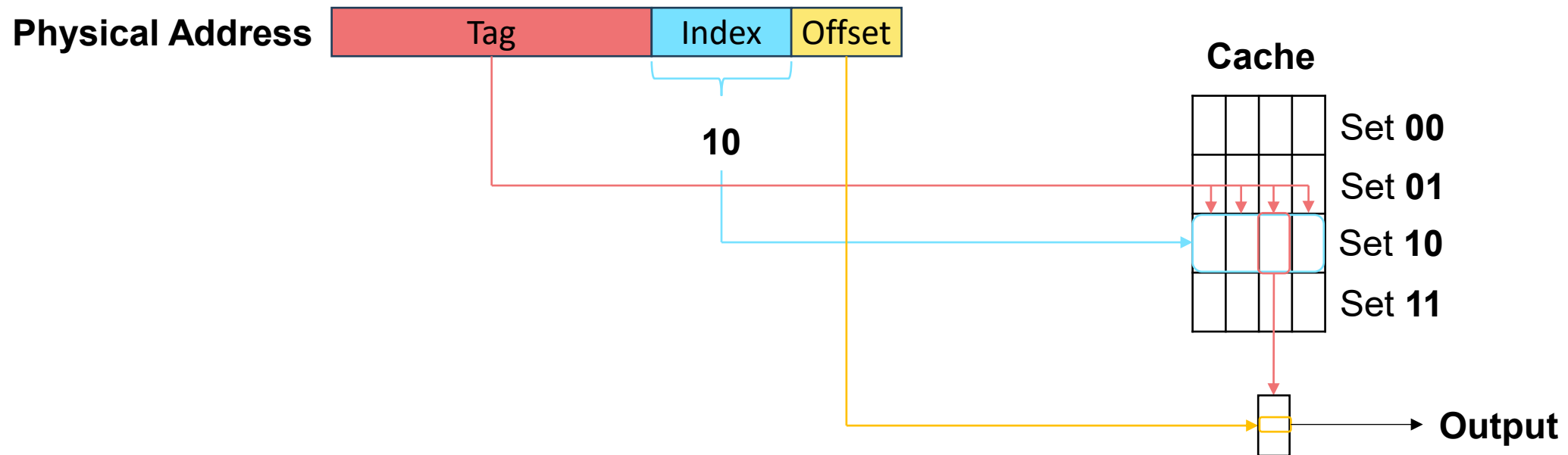
- Every entity resides on an access point, and the access point has an address
- Addresses may be location-dependent (or not)
- E.g., IP Address + Port

c) Identifier

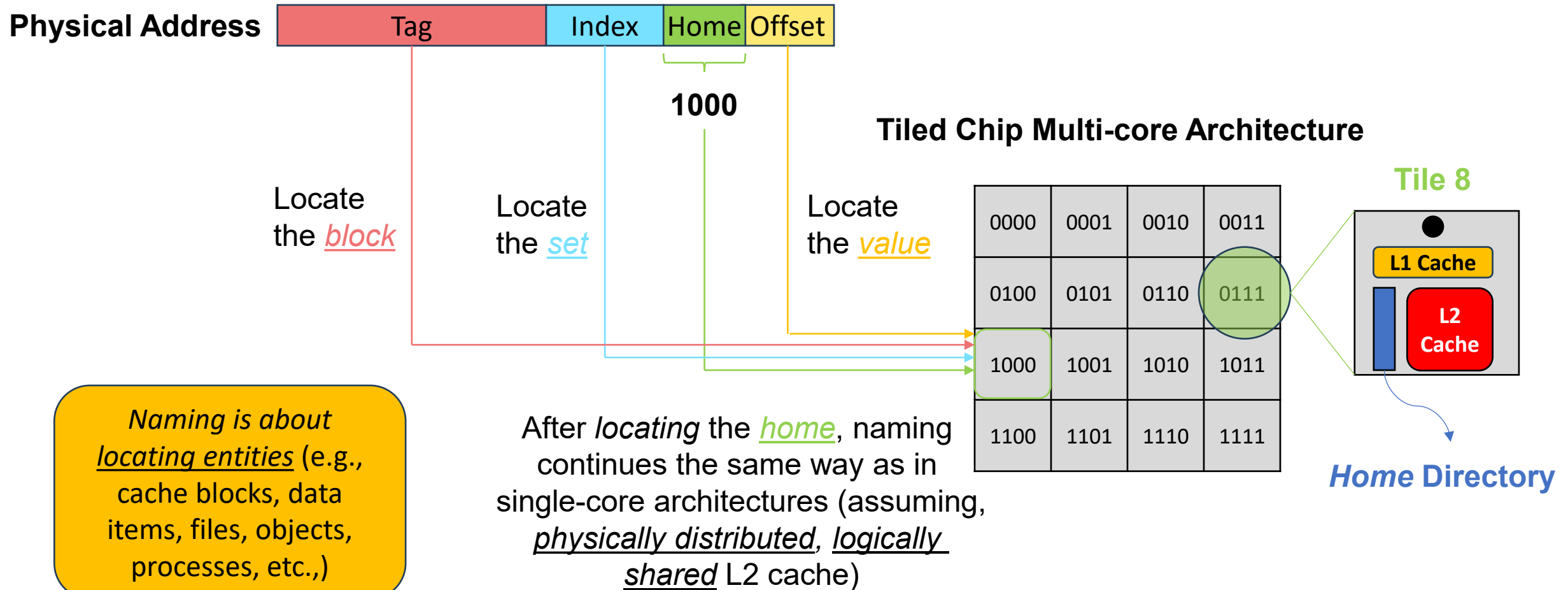
- Identifiers are names that *uniquely* identify entities
- A *true identifier* is a name with the following properties:
 - An identifier refers to at-most one entity
 - Each entity is referred to by at-most one identifier
 - An identifier always refers to the same entity (i.e. it is never reused)



Naming as We Know it from 15-213: An Example Assuming a Single-Core Architecture



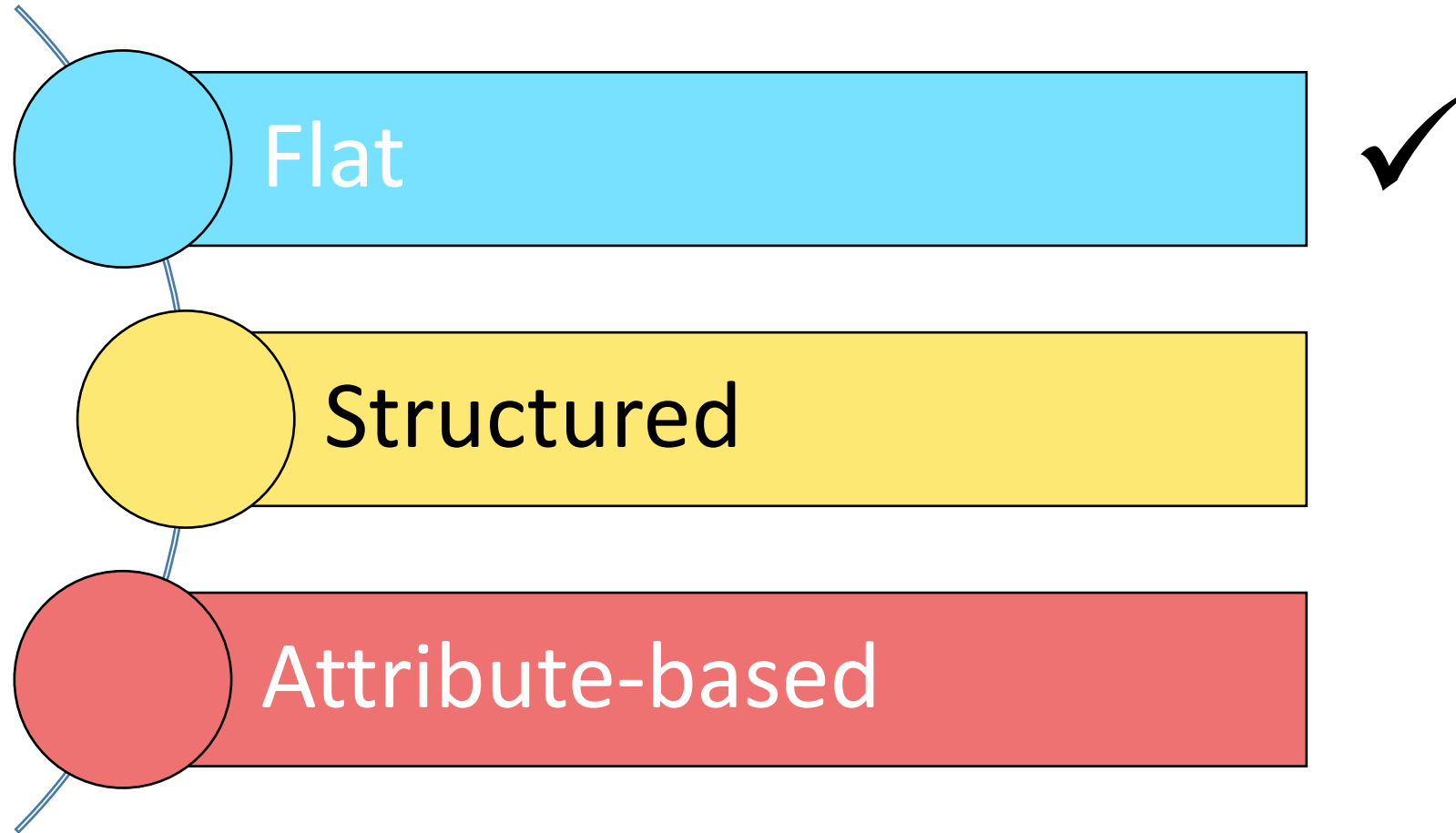
Naming as We Know it from 15-418: An Example Assuming a Multi-Core Architecture



Naming Systems as We Will Learn them in 15-440

- A *naming system* is a *middleware* that assists in *name resolution*
- Naming systems can be classified into three classes, based on the type of names used:
 - a. Flat naming
 - b. Structured naming
 - c. Attribute-based naming

Classes of Naming



Flat Naming

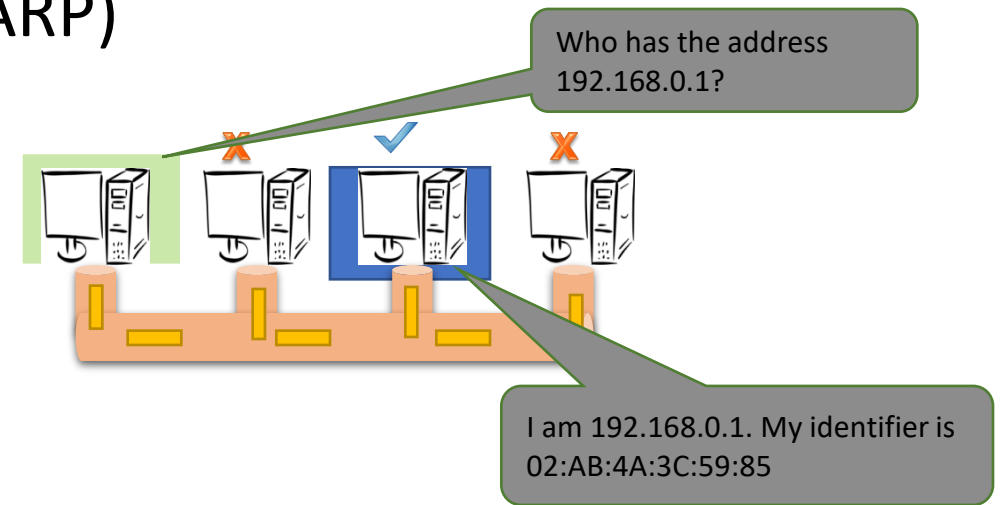
- In flat naming, identifiers are simply random bits of strings (known as *unstructured* or flat names)
- A flat name does not contain any information on how to locate an entity
- We will study four types of name resolution mechanisms for flat names:
 1. Broadcasting
 2. Forwarding pointers
 3. Home-based approaches
 4. Distributed Hash Tables (DHTs)

1. Broadcasting

- Approach: Broadcast the name/address to the whole network; the entity associated with the name responds with its current identifier

- Example: Address Resolution Protocol (ARP)

- Resolve an IP address to a MAC address
- In this system,
 - The IP address is the *address* of the entity
 - The MAC address is the *identifier* of the access point



- Challenges:

- Not scalable in large networks
 - This technique leads to flooding the network with broadcast messages
- Requires all entities to *listen* (or *snoop*) to all requests

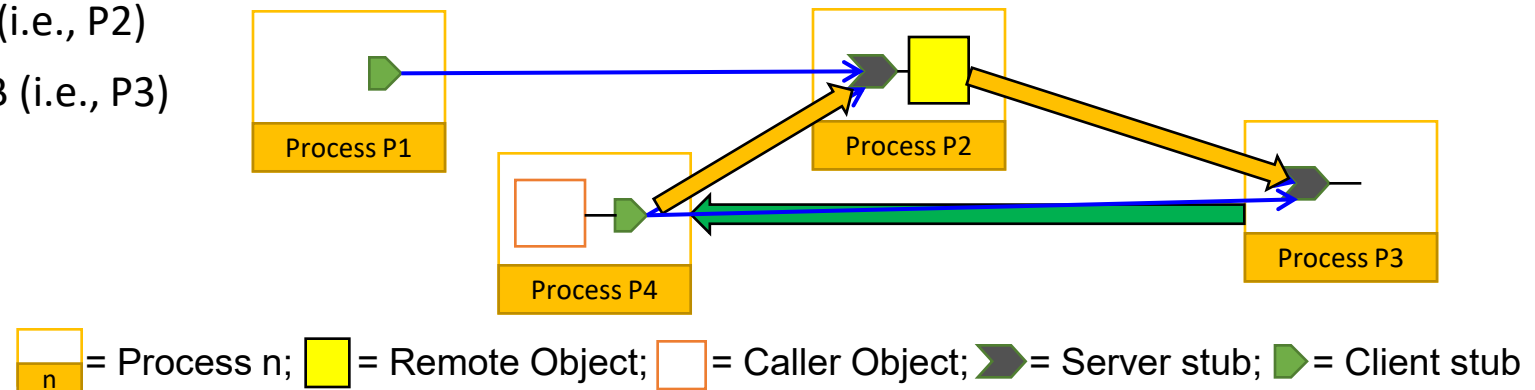
2. Forwarding Pointers

- Forwarding pointers enable locating *mobile* entities
 - Mobile entities move from one access point to another
- When an entity moves from location A to location B, it leaves behind (at A) a reference to its new location at B
- Name resolution mechanism:
 - Follow the *chain of pointers* to reach the entity
 - Update the entity's reference when the present location is found
- Challenges:
 - Long chains lead to longer resolution delays
 - Long chains are prone to failures due to broken links



Forwarding Pointers – An Example

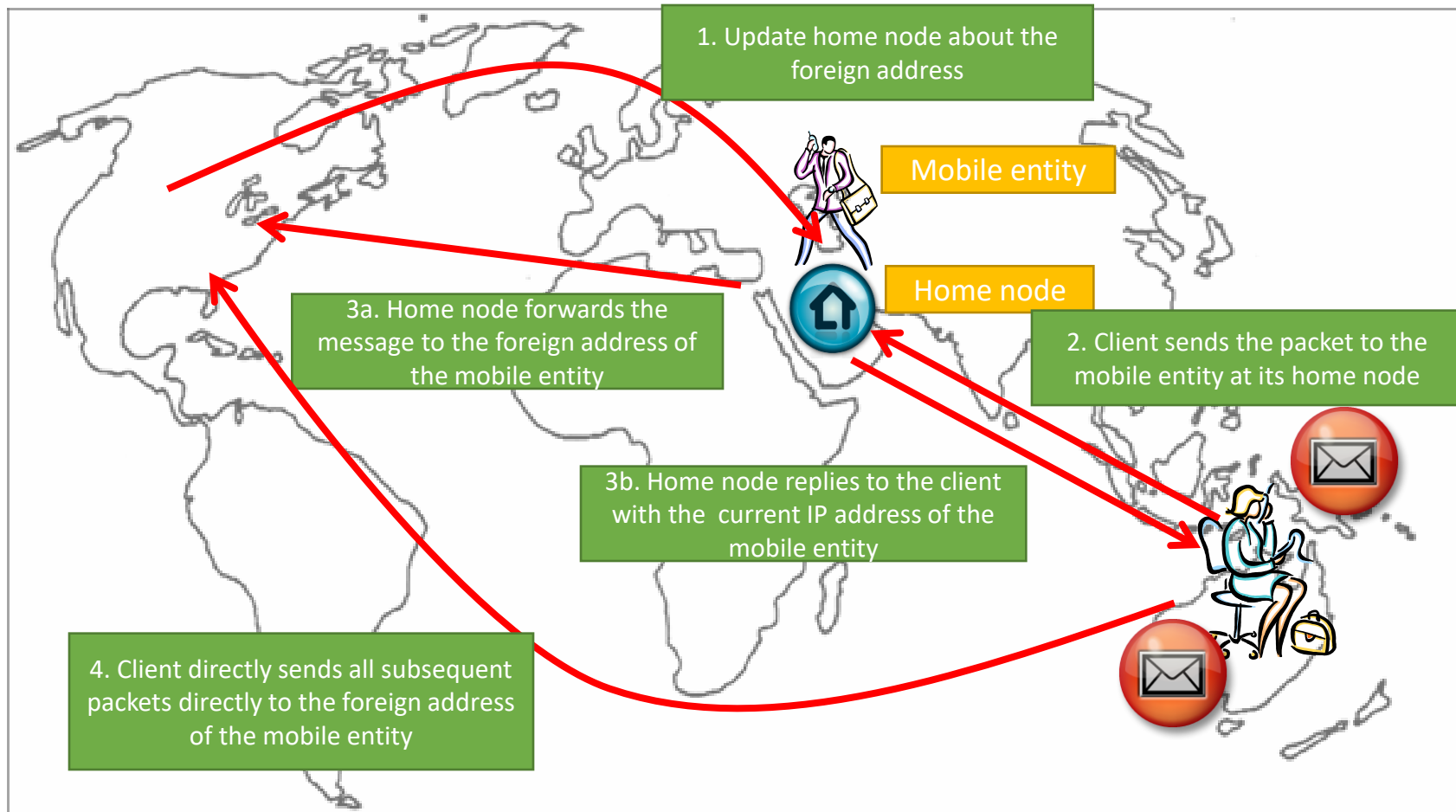
- Stub-Scion Pair (SSP) chains implement remote invocations for mobile entities using *forwarding pointers*
 - Server stub is referred to as Scion in the original paper
- Each forwarding pointer is implemented as a pair:
(client stub, server stub)
 - The server stub contains a local reference to the actual object or a local reference to another client stub
- When an object moves from A (e.g., P2) to B (e.g., P3),
 - It leaves a client stub at A (i.e., P2)
 - It installs a server stub at B (i.e., P3)



3. Home-Based Approaches

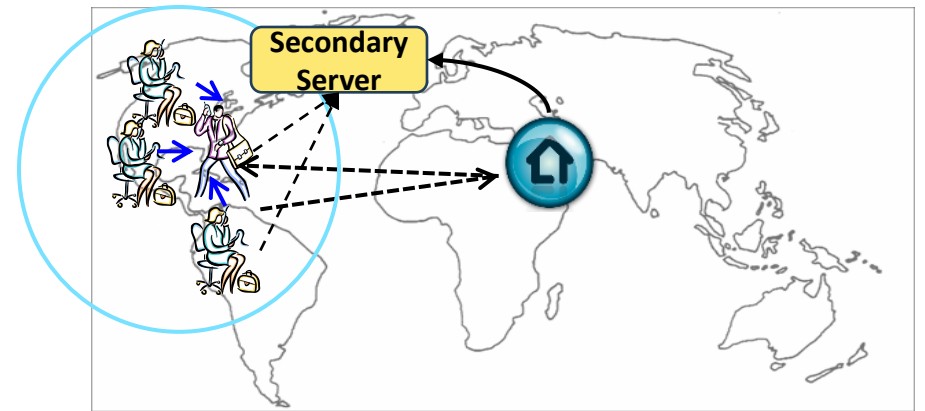
- Each entity is assigned a *home* node
 - The home node is typically static (has fixed access point and address)
 - It keeps track of the *current* address of the entity
- Entity-home interaction:
 - Entity's home address is registered at a naming service
 - The entity updates the home about its current address (*foreign address*) whenever it moves
- Name resolution:
 - The client contacts the home to obtain the foreign address
 - Afterwards, the client contacts the entity at the foreign location

3. Home-Based Approaches – An Example



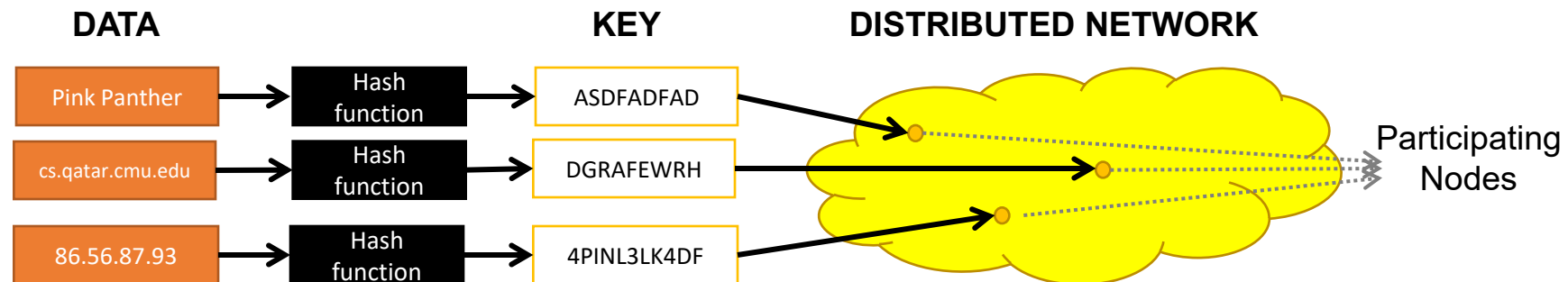
3. Home-Based Approaches – Challenges

- The static home address is permanent for an entity's lifetime
 - If the entity moves permanently, a *simple* home-based approach incurs higher communication overhead
 - Consider the scenario where the clients are nearer to the mobile entity than the home entity
 - *Caching* the address of the entity at the clients' sides can help, unless the entity moves very frequently
 - How to handle *invalid* addresses?
 - *Replicating* the address of the entity at a *secondary server* can help
 - How to handle *inconsistent* addresses?
 - Adopting *a dynamic home* for the entity can help (with or without replication)



4. Distributed Hash Table (DHT)

- DHT is a distributed system that provides a lookup service similar to a hash table
 - *(key, value)* pairs are stored in the nodes participating in the DHT
 - The responsibility for maintaining the mapping from keys to values is distributed among the nodes
 - Any participating node can serve in retrieving the value for a given key
- We will study a representative DHT known as **Chord**



Chord

- Chord assigns an *m-bit identifier* (randomly chosen) to each node
 - A node can be contacted through its network address
- Furthermore, it maps each entity to a node
 - Entities can be processes, files, etc.,
- Mapping of entities to nodes
 - Each node is responsible for a set of entities
 - An entity with **key** k falls under the jurisdiction of the node with the *smallest identifier* $id \geq k$. This node is known as the *successor of* k , and is denoted as ***succ* (k)**

Entity
with k

Node n (node
with $id=n$)

000

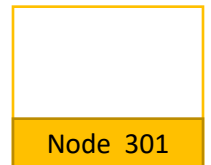
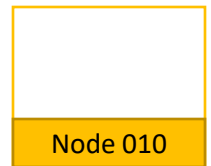
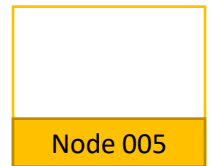
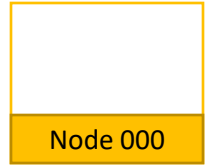
003

004

008

040

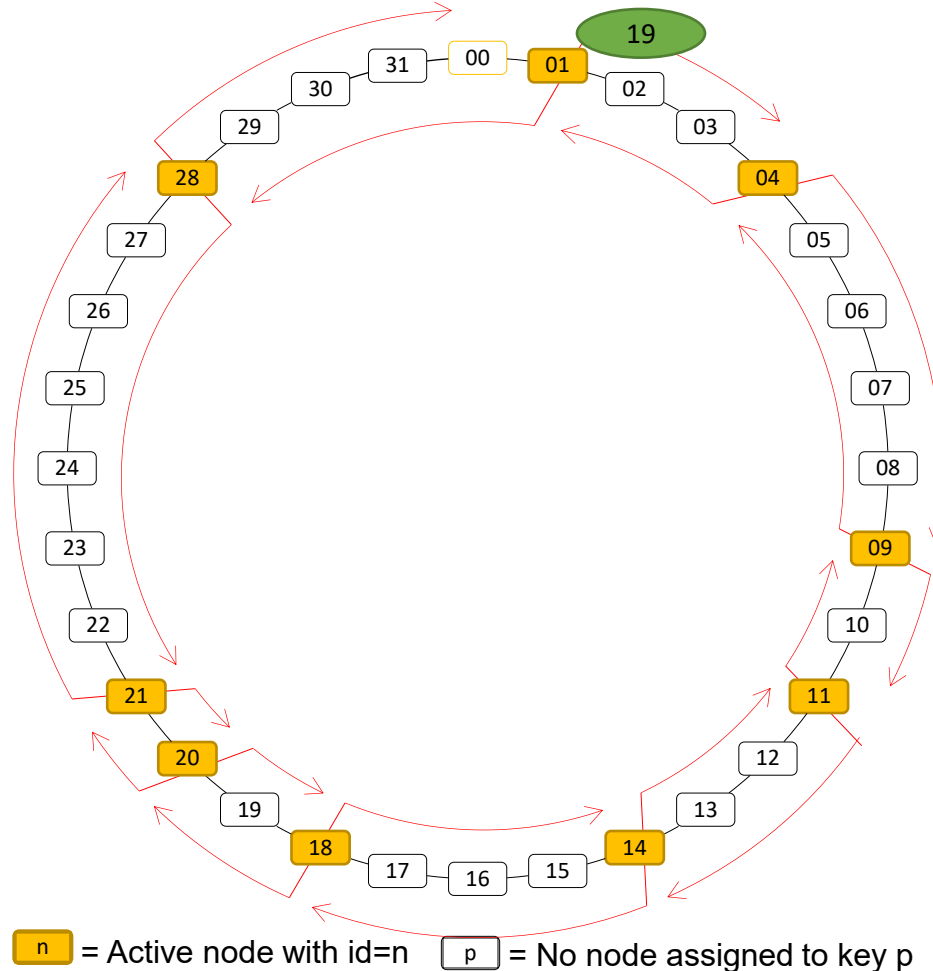
079



Map each entity with key k to
node $succ(k)$

A Naïve Key Resolution Algorithm

- The main objective in DHT is to efficiently resolve a key k to the network location of $\text{succ}(k)$
 - Given an entity with key k , how to find the node $\text{succ}(k)$?

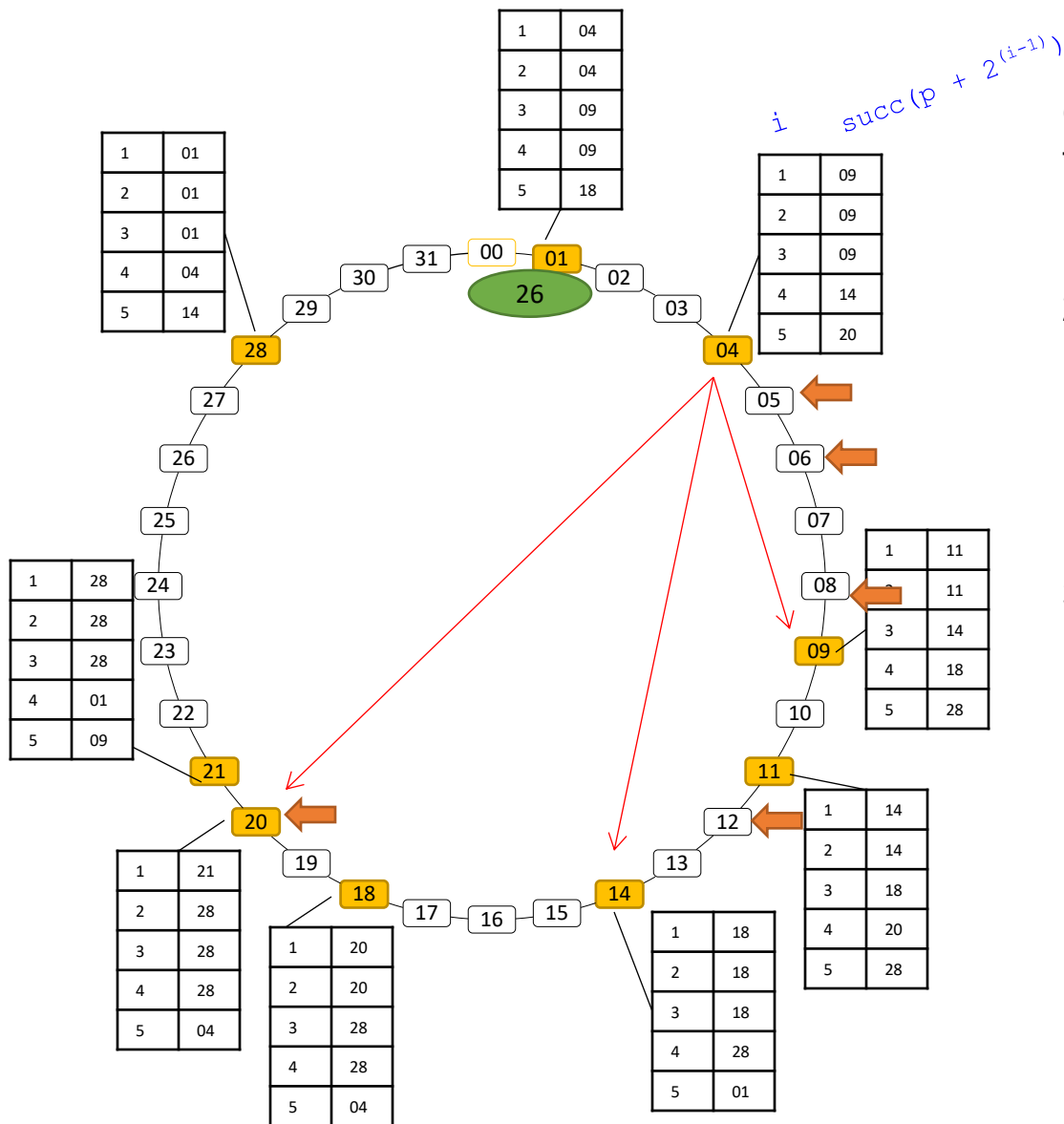


- All nodes are arranged in a logical ring according to their IDs
- Each node 'p' keeps track of its immediate neighbors: $\text{succ}(p)$ and $\text{pred}(p)$
- If 'p' receives a request to resolve key 'k':
 - If $\text{pred}(p) < k \leq p$, node p will handle it
 - Else it will forward it to $\text{succ}(n)$ or $\text{pred}(n)$

Solution is not scalable:

- As the network grows, forwarding delays increase
- Key resolution has a time complexity of $O(n)$

Key Resolution in Chord



Chord improves key resolution by reducing the time complexity to $O(\log n)$

1. All nodes are arranged in a logical ring according to their IDs
2. Each node 'p' keeps a table FT_p of at-most m entries. This table is called Finger Table

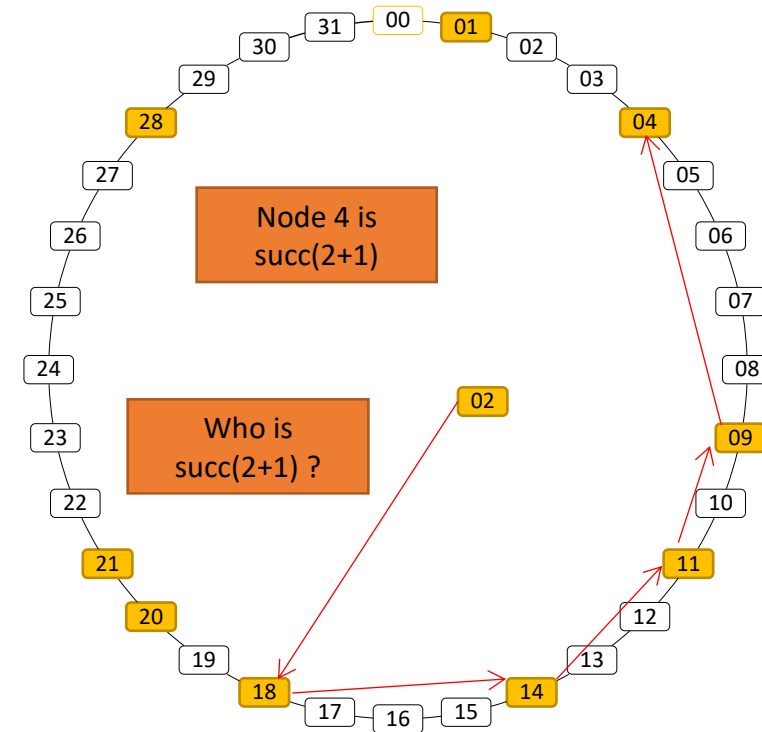
$$FT_p[i] = \text{succ}(p + 2^{(i-1)})$$

NOTE: $FT_p[i]$ increases exponentially

3. If node 'p' receives a request to resolve key 'k':
 - Node p will forward it to node q with index j in F_p where $q = FT_p[j] \leq k < FT_p[j+1]$
 - If $k > FT_p[m]$, then node p will forward it to $FT_p[m]$
 - If $k < FT_p[1]$, then node p will forward it to $FT_p[1]$

Chord – The Join and Leave Protocol

- In large-scale distributed Systems, nodes dynamically *join* and *leave* (voluntarily or due to failures)
- If a node p wants to join:
 - It contacts an arbitrary node, looks up for $\text{succ}(p+1)$, and inserts itself into the ring
- If node p wants to leave:
 - It contacts $\text{pred}(p)$ and $\text{succ}(p+1)$ and updates them



Chord – The Finger Table Update Protocol

- For any node q , $FT_q[1]$ should be up-to-date
 - It refers to the next node in the ring
 - Protocol:
 - Periodically, request $\text{succ}(q+1)$ to return $\text{pred}(\text{succ}(q+1))$
 - If $q = \text{pred}(\text{succ}(q+1))$, then information is up-to-date
 - Otherwise, a new node p has been added to the ring such that $q < p < \text{succ}(q+1)$
 - $FT_q[1] = p$
 - Request p to update $\text{pred}(p) = q$
 - Similarly, node p updates each entry i by finding $\text{succ}(p + 2^{(i-1)})$

Exploiting Network Proximity in Chord

- The logical organization of nodes in the overlay network may lead to inefficient message transfers
 - Node k and node $\text{succ}(k + 1)$ may be far apart
- Chord can be optimized by considering the network location of nodes
 1. Topology-Aware Node Assignment
 - Two nearby nodes get identifiers that are close to each other
 2. Proximity Routing
 - Each node q maintains ' r ' successors for i^{th} entry in the finger table
 - $\text{FT}_q[i]$ now refers to r successor nodes in the range $[p + 2^{(i-1)}, p + 2^i - 1]$
 - To forward the lookup request, pick one of the r successors closest to the node q



Next Lecture...

- Structured and attribute-based namings

Today...

- Last Session:

- Naming- Part I
 - Flat naming

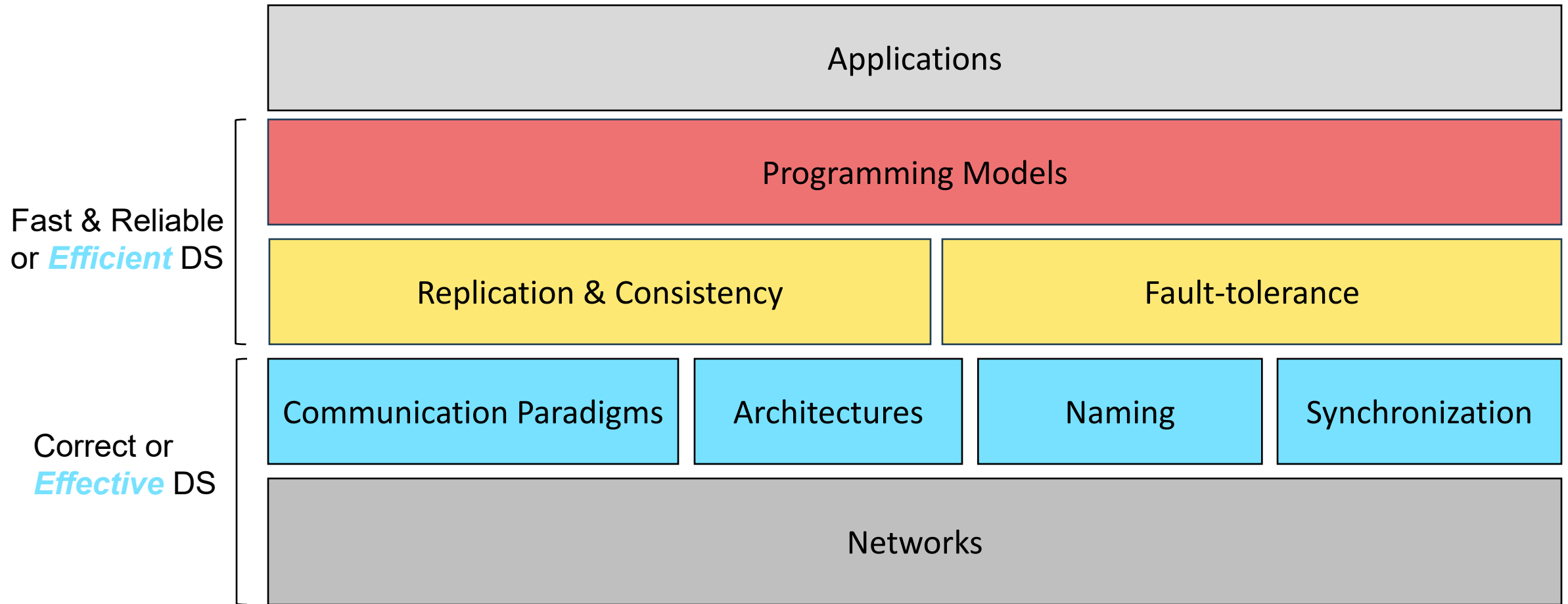
- Today's Session:

- Naming- Part II
 - Structured and attribute-based namings

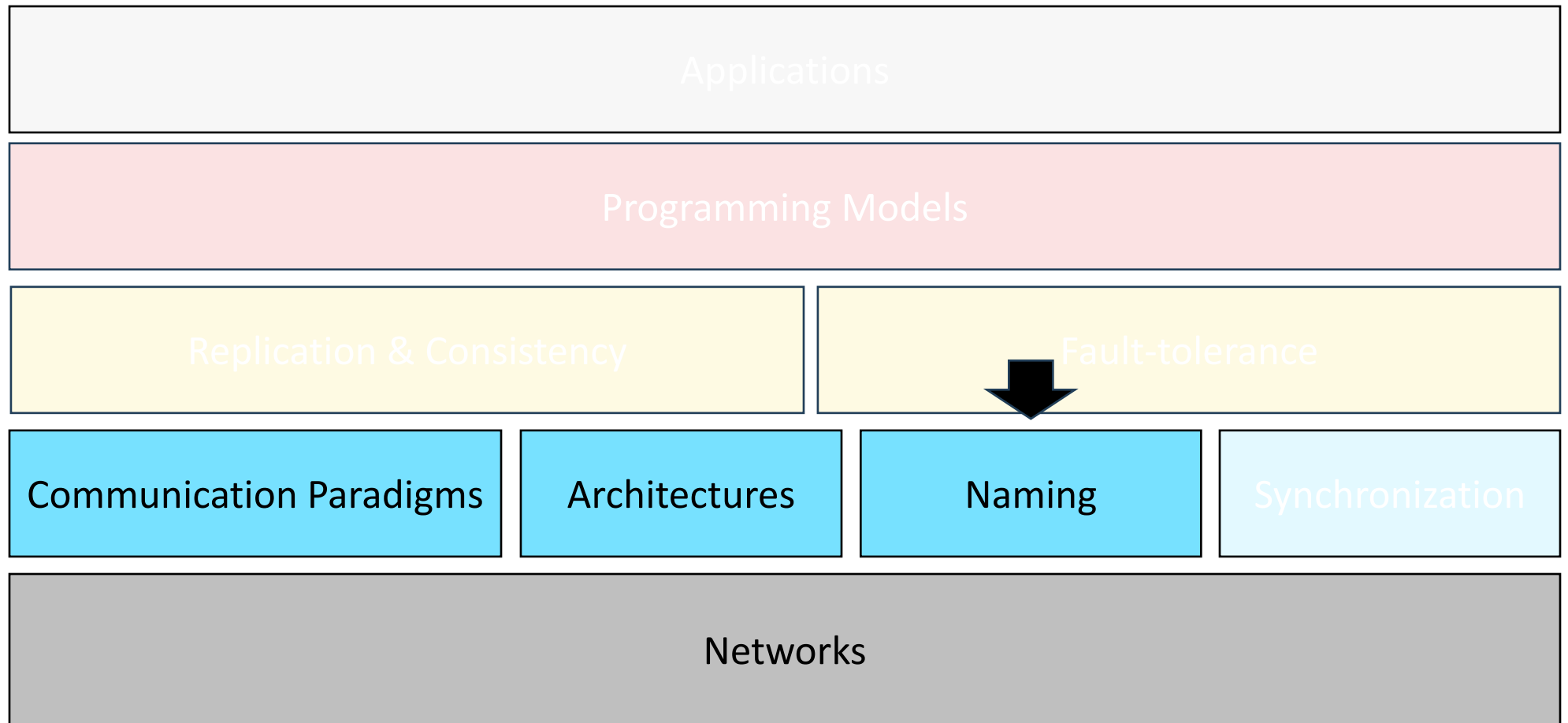
- Announcements:

- PS2 is due on Oct 1st
- Quiz 2 is on Sunday, September 29th

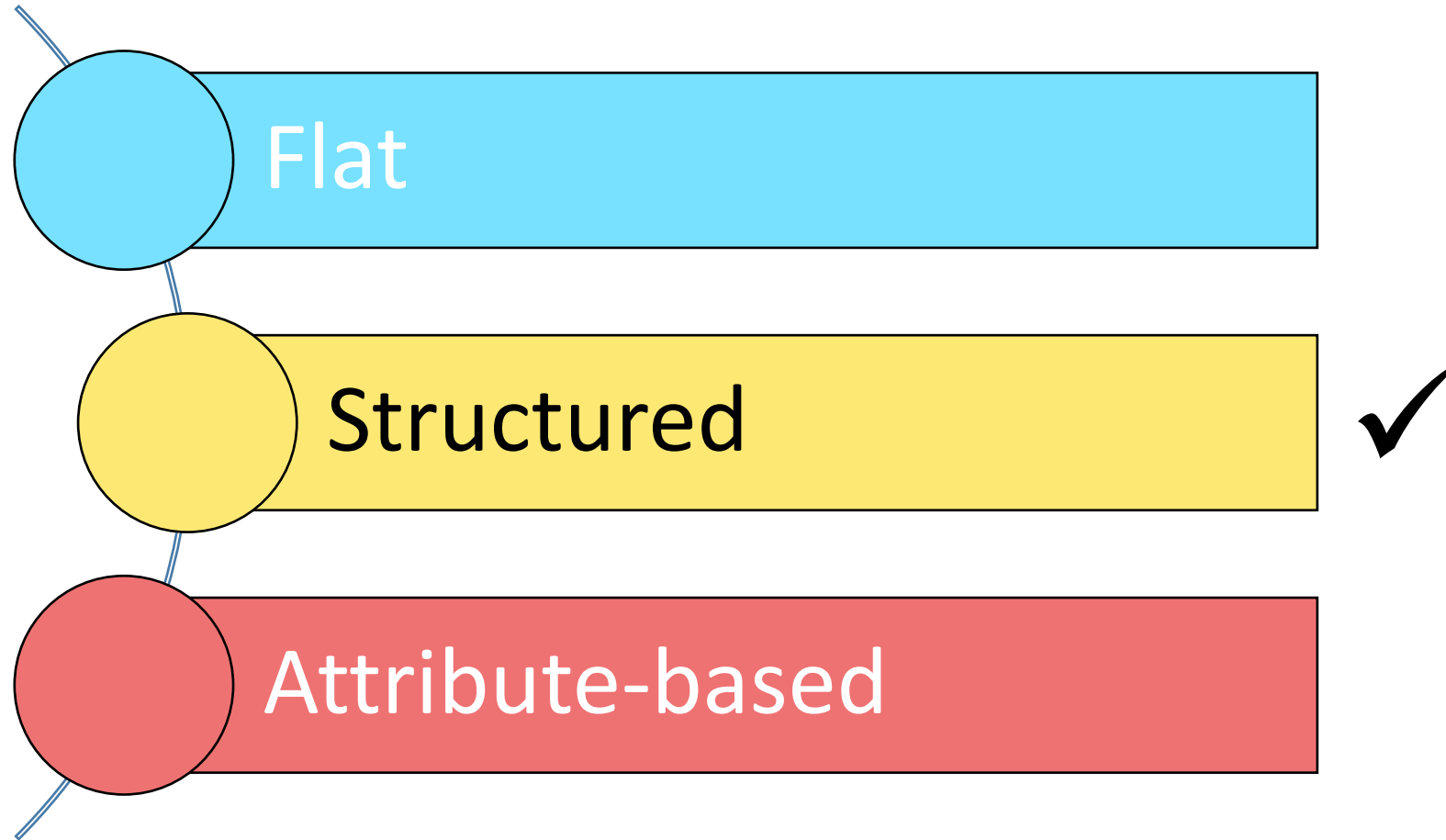
Course Map



Course Map



Classes of Naming



Structured Naming

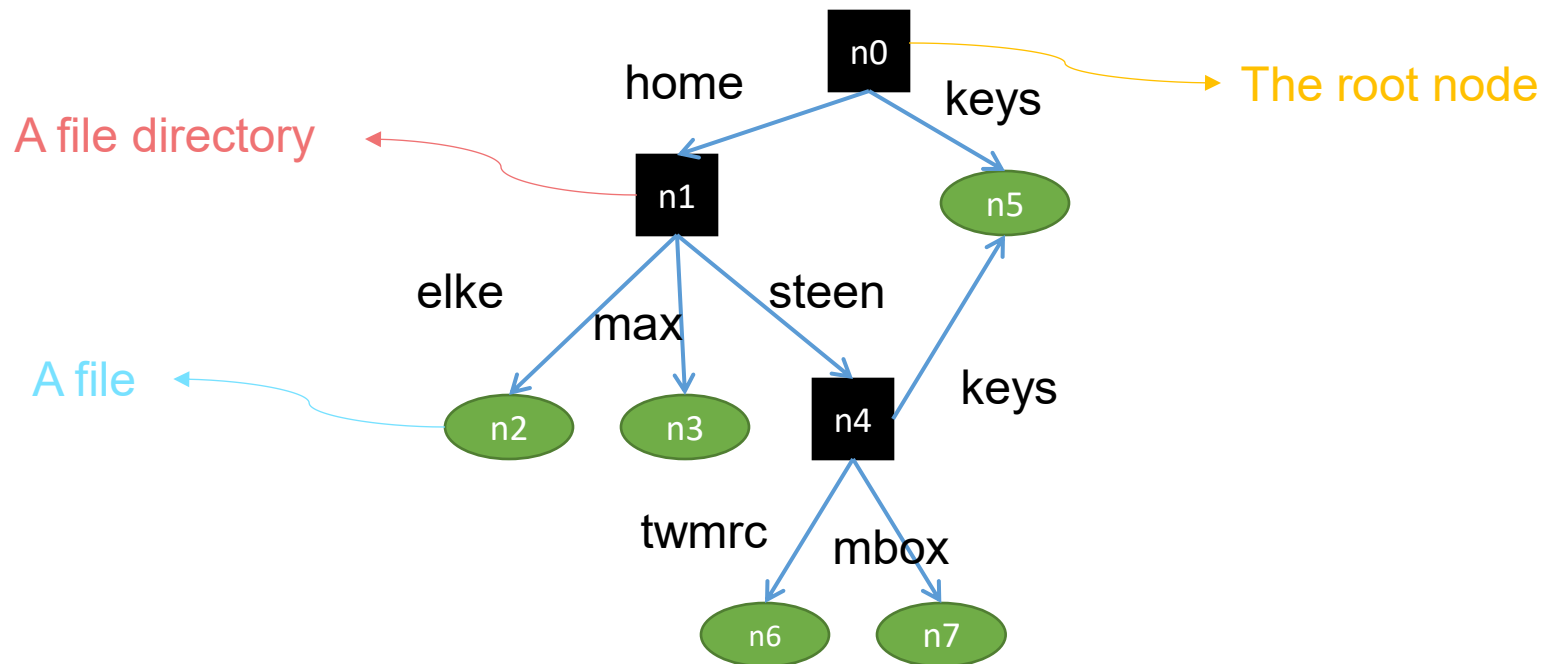
- Structured names are composed of simple human-readable names, with certain structures based on the contexts
- **Examples:**
 - In the context of a file-system
 - /home/userid/work/dist-systems/naming.txt is the structured name of the file naming.txt
 - In the context of a website
 - <https://inelpandzic.com/articles/system-models-distributed-systems/> is the structured name of the 15440 webpage

Name Spaces

- Structured names are organized into *name spaces*
- A name space is a *directed graph* (referred to as a *naming graph*) consisting of:
 - **Leaf nodes**
 - Represent entities (e.g., files or webpages)
 - Generally store the addresses (e.g., as in DNS), the states (e.g., contents of files in file systems), or the absolute path names (e.g., to other nodes) of the entities they represent
 - **Directory nodes**
 - Each directory node refers to other leaf or directory node(s)
 - Each outgoing edge is represented by (*edge label*, *destination node identifier*)

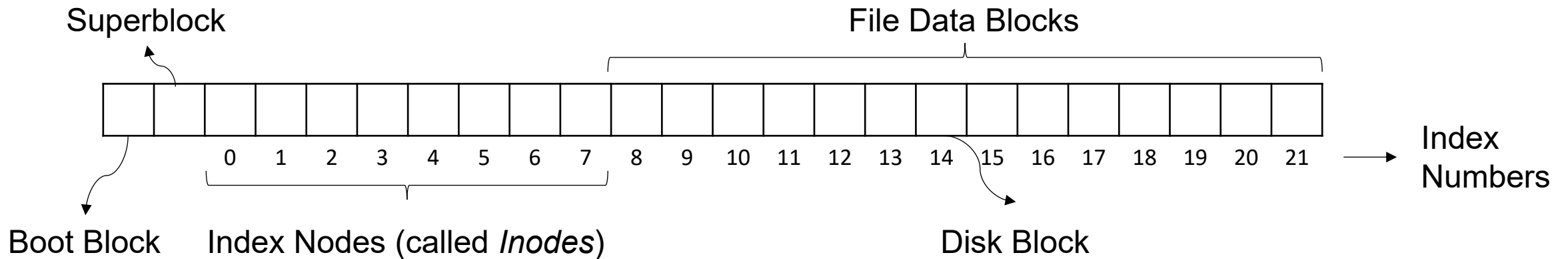
Example: The Unix Naming Graph

- In the naming graph of Unix
 - A directory node represents a *file directory*
 - A leaf node represents a *file*
 - There is a single root directory, represented by the *root node*



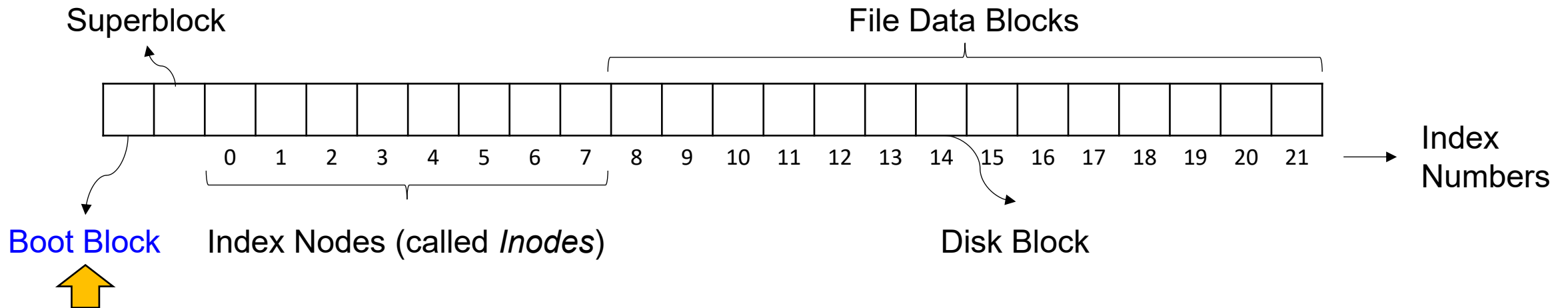
Example: The Unix Naming Graph

- The implementation (or the *physical representation*) of the Unix naming graph consists of a *contiguous series of blocks* in a disk



Example: The Unix Naming Graph

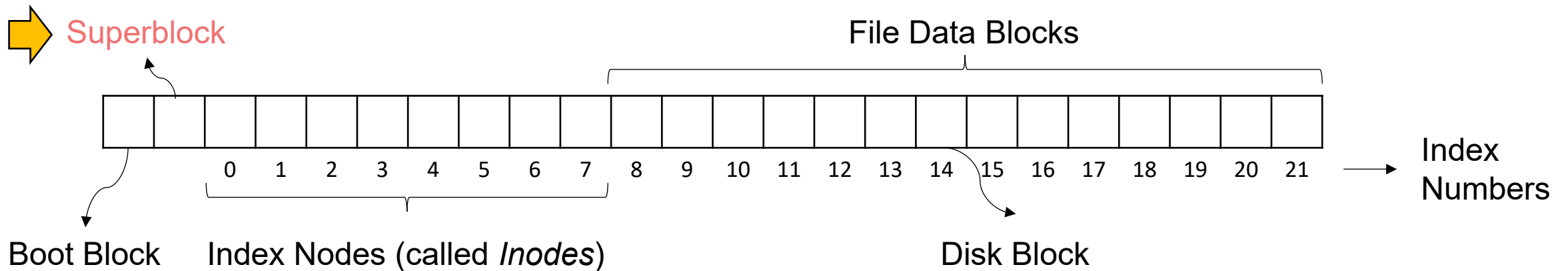
- The implementation (or the *physical representation*) of the Unix naming graph consists of a *contiguous series of blocks* in a disk



- The **Boot Block** is a special block of data and instructions that are automatically loaded into main memory when the system is booted
- It is used to load the OS into main memory

Example: The Unix Naming Graph

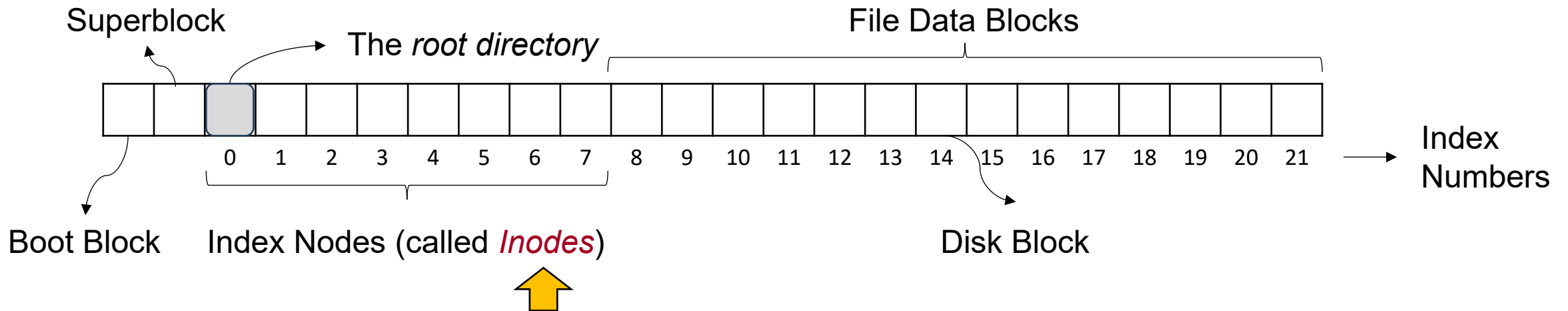
- The implementation (or the *physical representation*) of the Unix naming graph consists of a *contiguous series of blocks* in a disk



- The **Superblock** contains information on the entire file system (e.g., its size, which blocks on disk are not yet allocated, which Inodes are not yet used, etc.,)

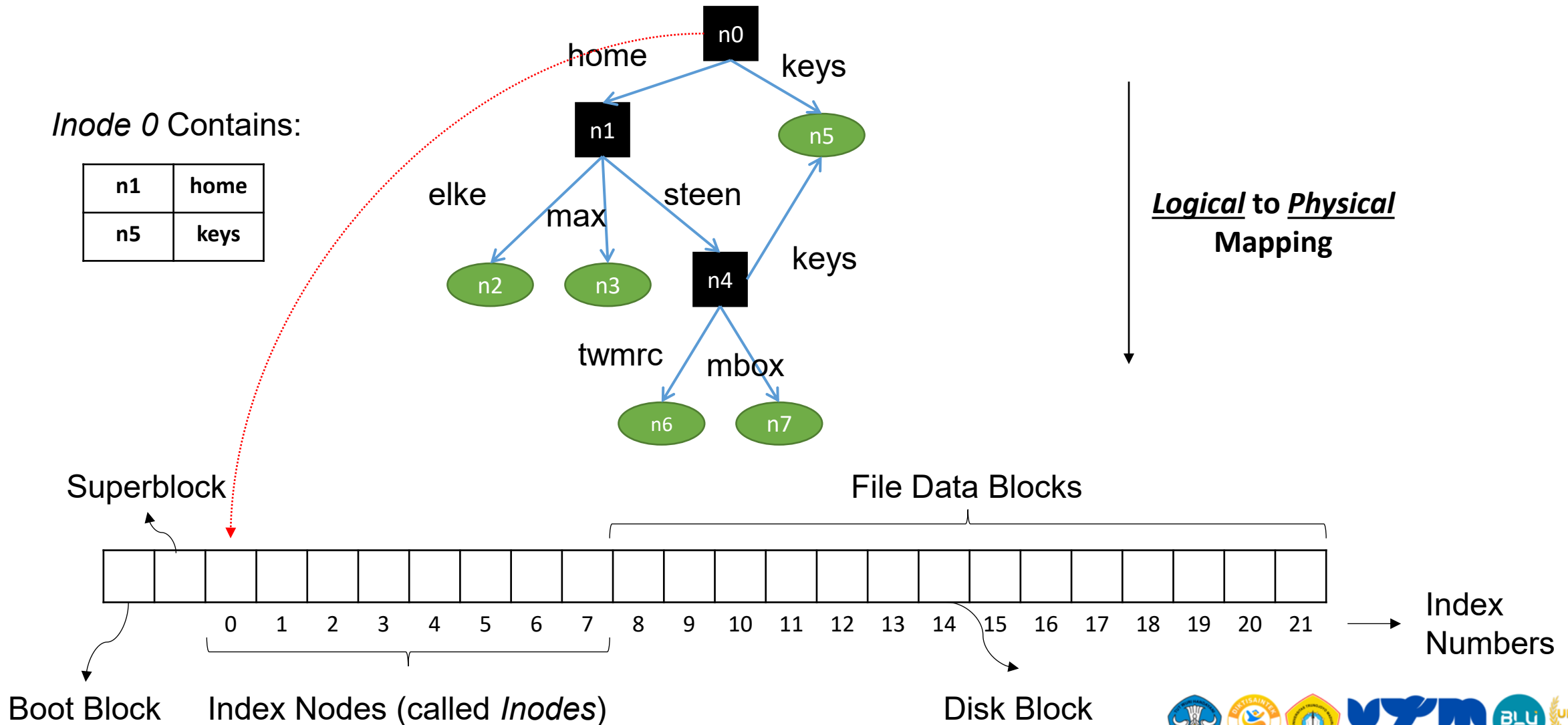
Example: The Unix Naming Graph

- The implementation (or the *physical representation*) of the Unix naming graph consists of a *contiguous series of blocks* in a disk



- The *Inodes* are referred to by index numbers, starting at zero, which is reserved for the Inode representing the *root directory*
- Each Inode contains information on *where* the data of its associated file can be found on disk
- Besides, an Inode contains information on the file's owner, time of creation and last modification, protection, etc.,

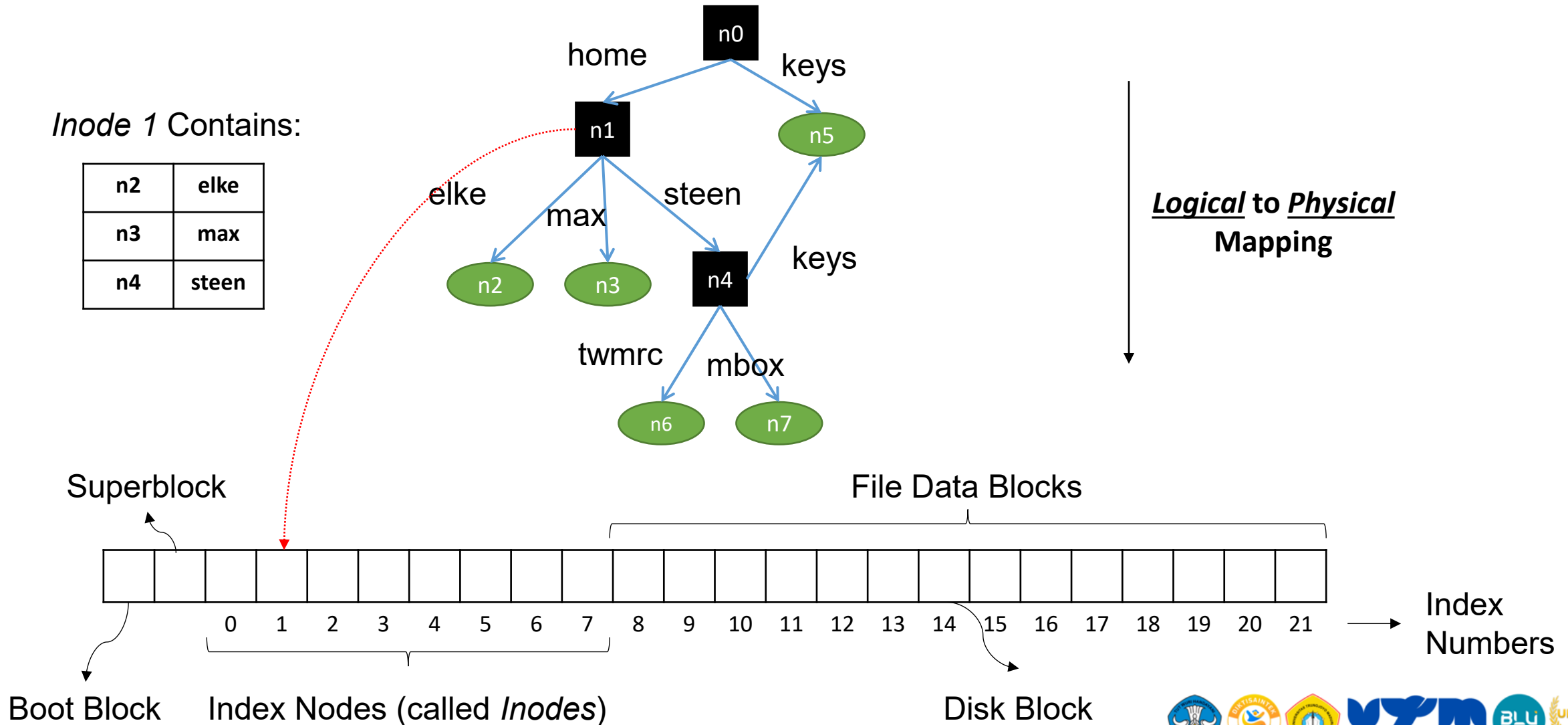
The Unix Naming Graph: Putting it Altogether



The Unix Naming Graph: Putting it Altogether

Inode 1 Contains:

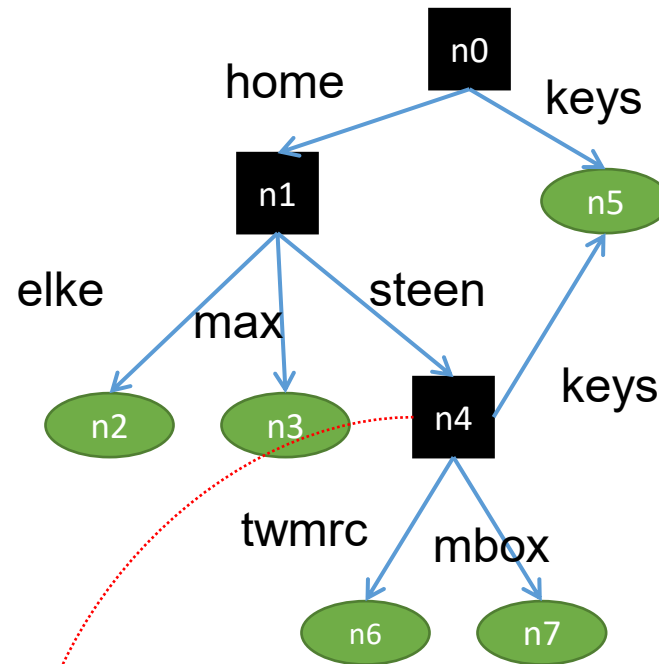
n2	elke
n3	max
n4	steen



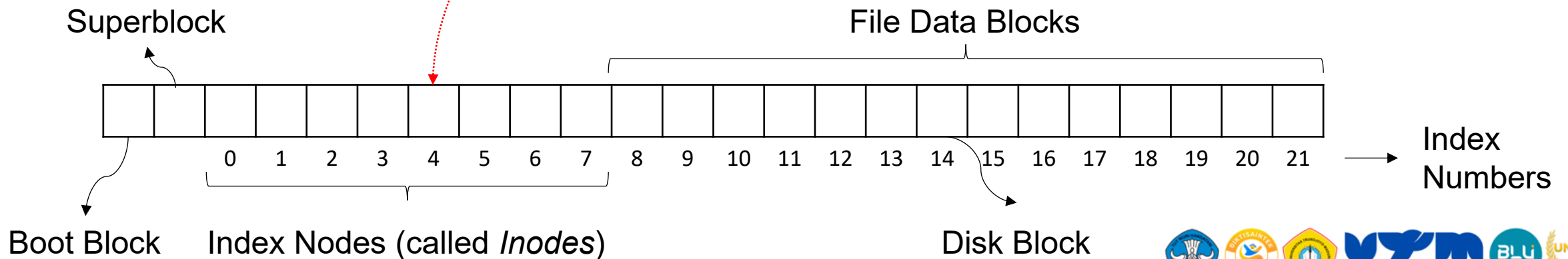
The Unix Naming Graph: Putting it Altogether

Inode 4 Contains:

n6	twmrc
n7	mbox
n5	keys



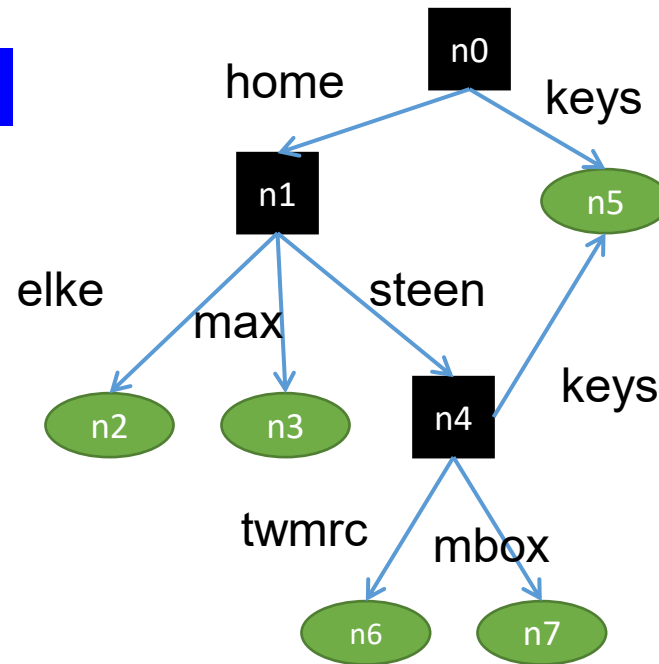
Logical to Physical
Mapping



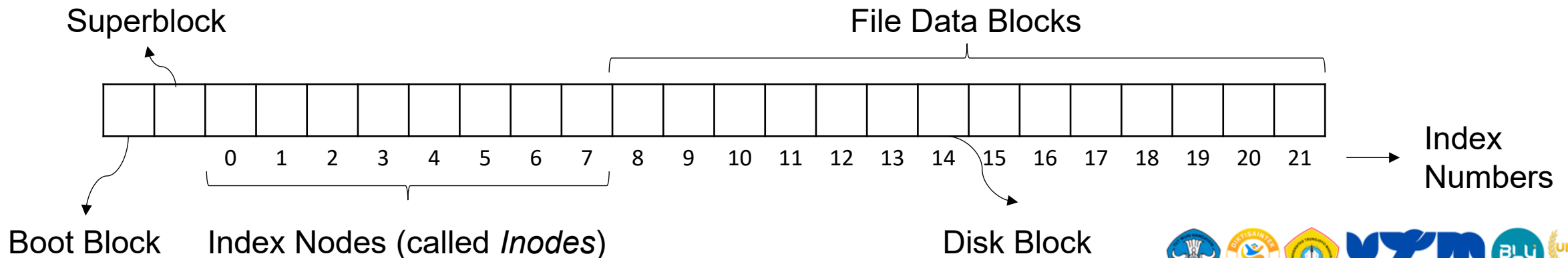
The Unix Naming Graph: Putting it Altogether

Lookup **/home/steen/mbox**

The process of looking up a name (e.g., /home/steen/mbox) is called **name resolution**



Logical to Physical
Mapping



The Unix Naming Graph: Putting it Altogether

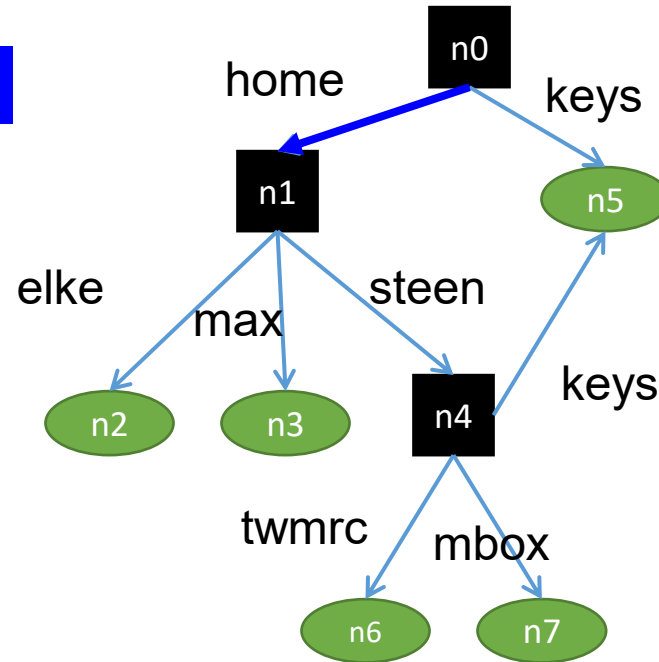


Lookup **/home/steen/mbox**

Inode 0 Contains:

n1	home
n5	keys

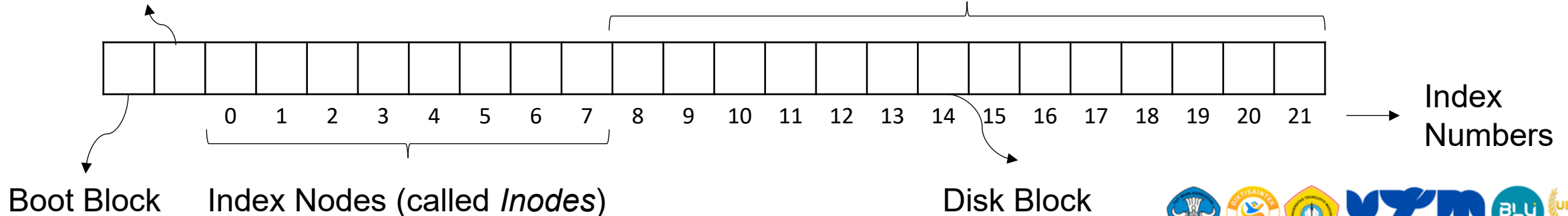
Through “home” we
get into n1 (from n0)



Logical to Physical
Mapping

Superblock

File Data Blocks



The Unix Naming Graph: Putting it Altogether

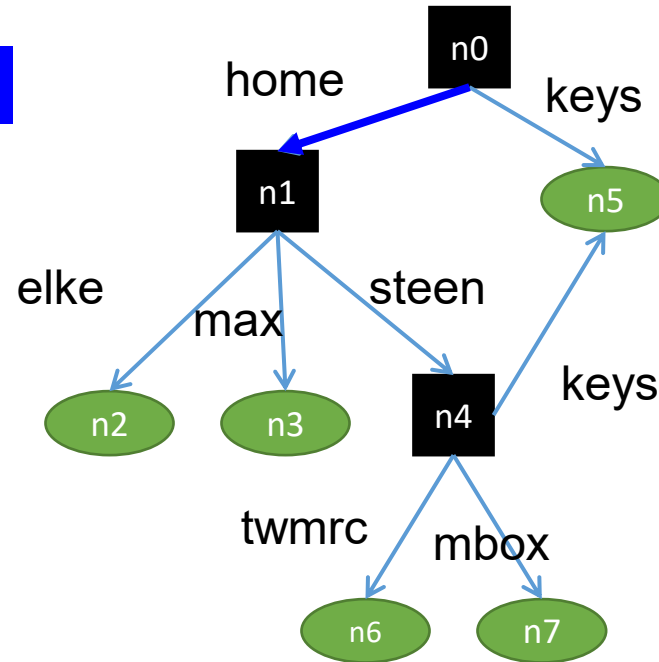


Lookup **/home/steen/mbox**

Inode 0 Contains:

n1	home
n5	keys

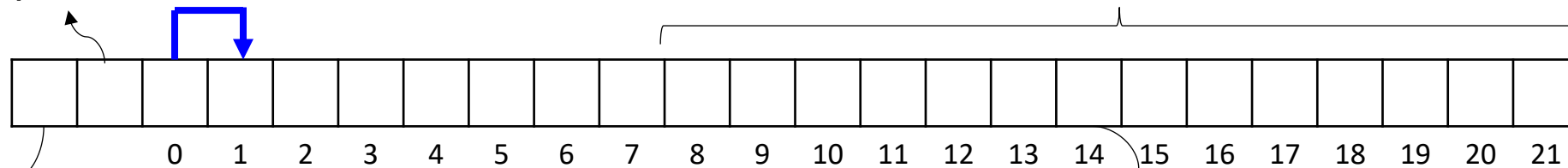
Through “home” we get into n1 (from n0)



Logical to Physical
Mapping

Superblock

File Data Blocks



Index
Numbers

Boot Block

Index Nodes (called *Inodes*)

Disk Block



The Unix Naming Graph: Putting it Altogether

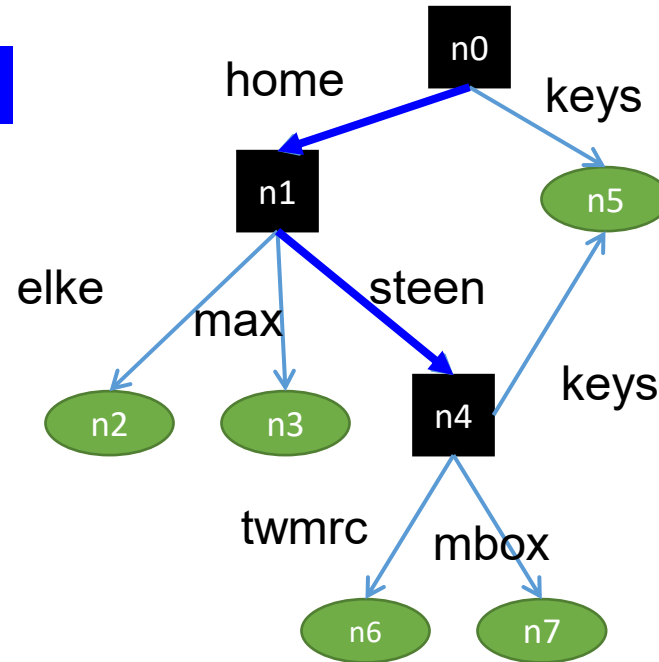


Lookup **/home/steen/mbx**

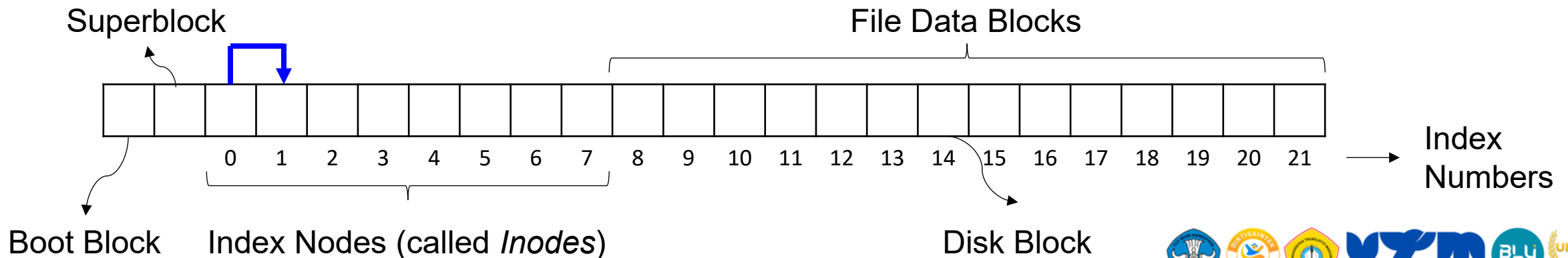
Inode 1 Contains:

n2	elke
n3	max
n4	steen

Through “steen” we
get into n4 (from n1)



Logical to Physical
Mapping



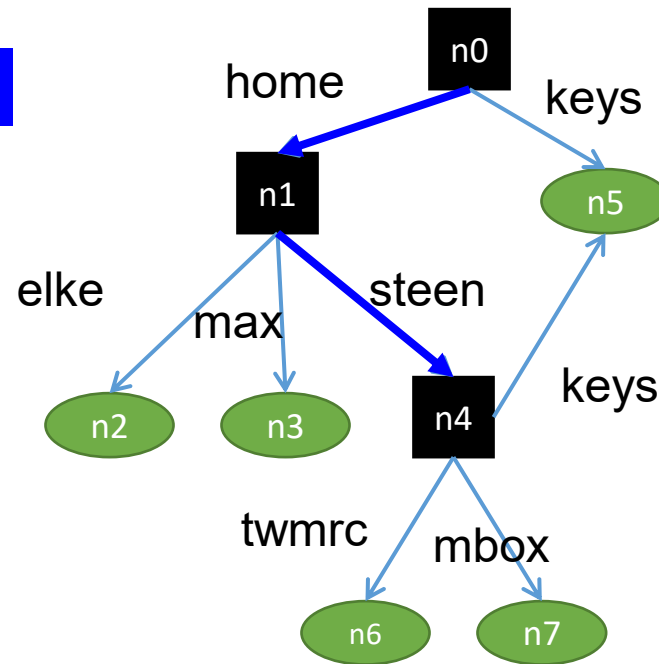
The Unix Naming Graph: Putting it Altogether

Lookup /home/steen/mbox

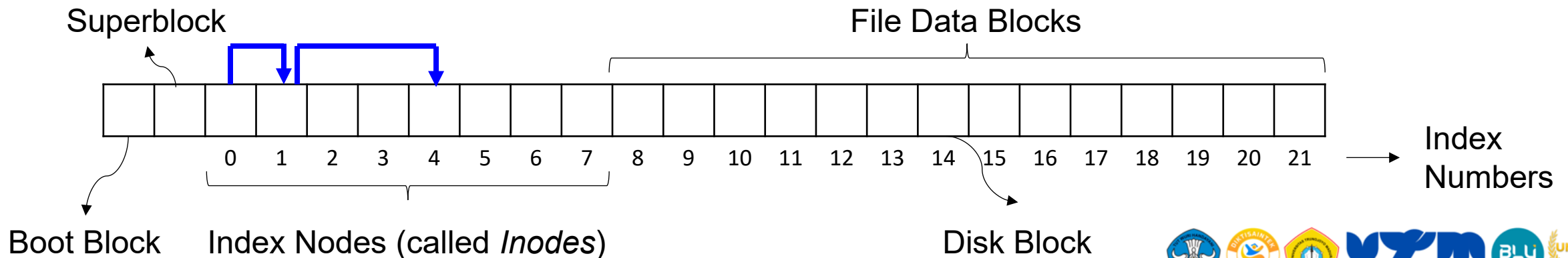
Inode 1 Contains:

n2	elke
n3	max
n4	steen

Through “steen” we get into n4 (from n1)



Logical to Physical
Mapping



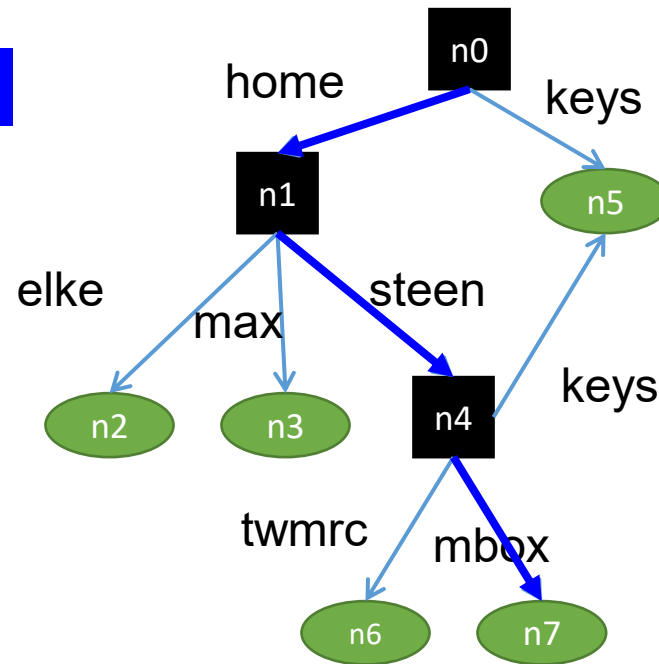
The Unix Naming Graph: Putting it Altogether

Lookup /home/steen/mbox

Inode 4 Contains:

n6	twmrc
n7	mbox
n5	keys

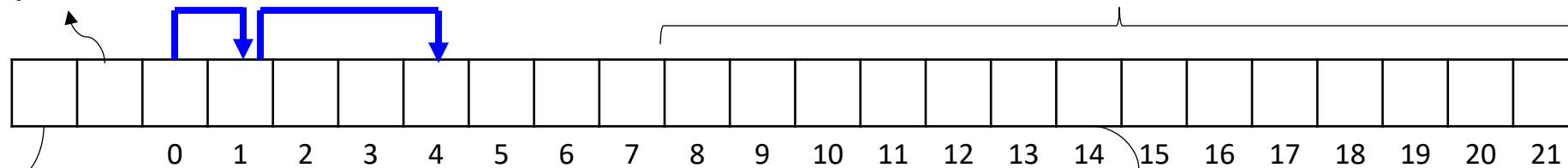
Through “mbox” we get into n7 (from n4)



Logical to Physical
Mapping

Superblock

File Data Blocks



Index
Numbers

Boot Block

Index Nodes (called *Inodes*)

Disk Block



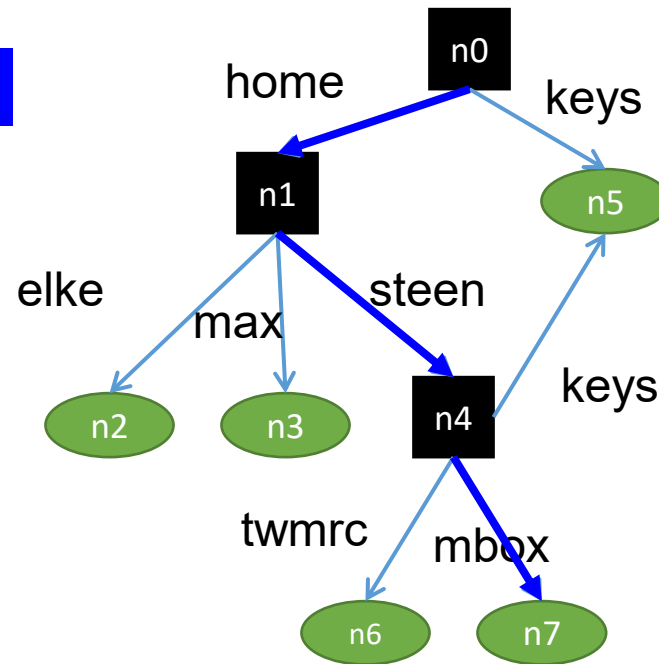
The Unix Naming Graph: Putting it Altogether

Lookup /home/steen/mbox

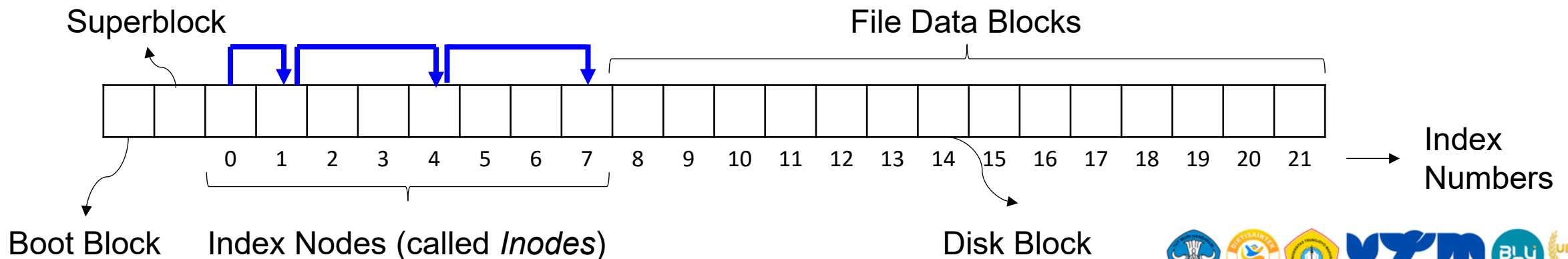
Inode 4 Contains:

n6	twmrc
n7	mbox
n5	keys

Through “mbox” we get into n7 (from n4)



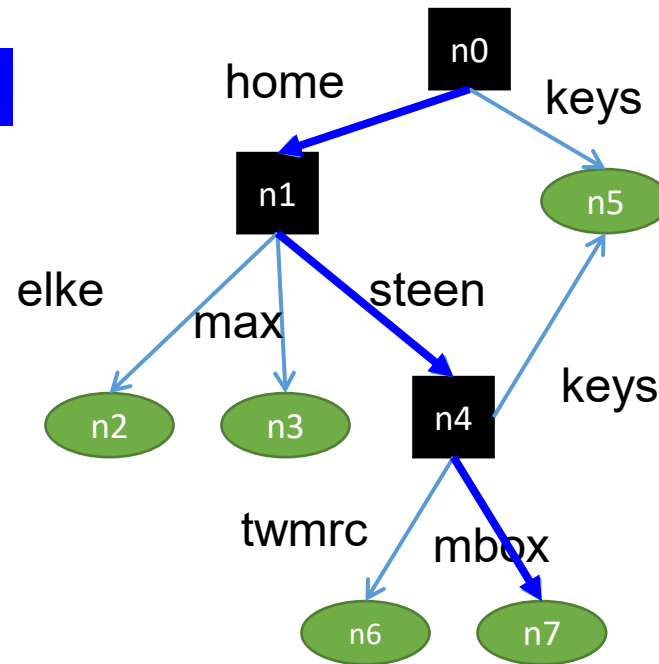
Logical to Physical
Mapping



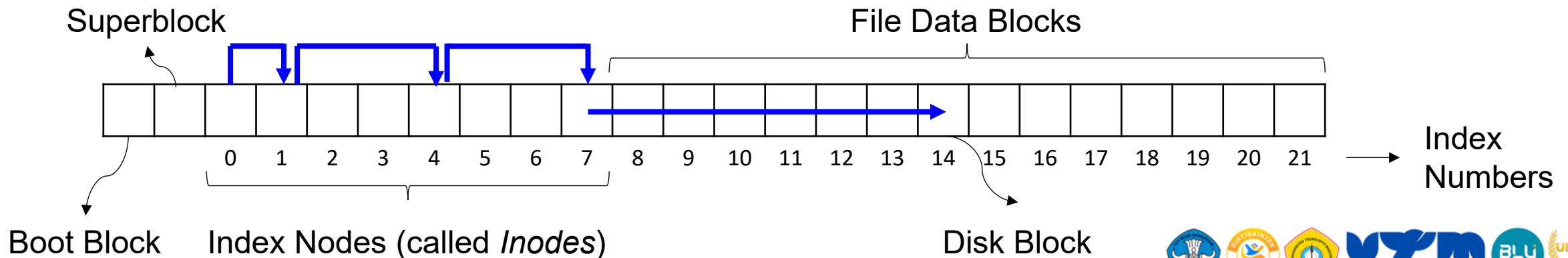
The Unix Naming Graph: Putting it Altogether

Lookup **/home/steen/mbox**

n7 is an Inode that contains information on where the *data* of mbox can be found on disk (say, at n14)



Logical to Physical
Mapping



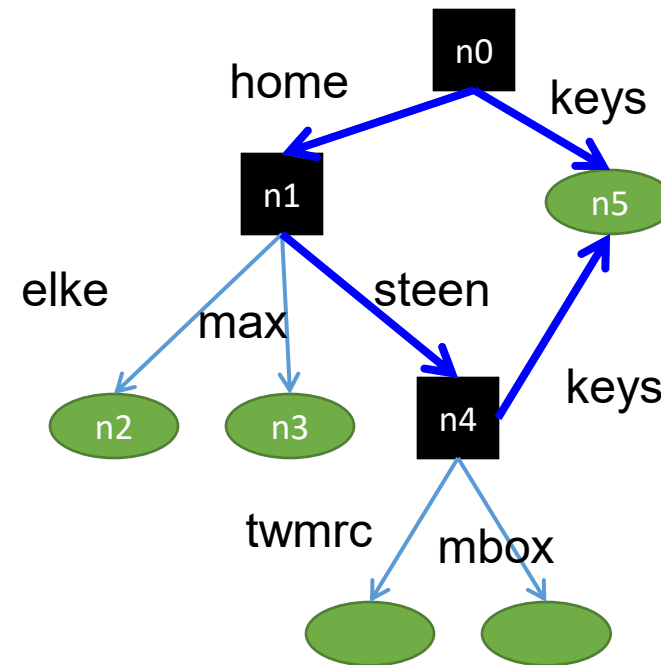
Name Linking

- Strongly related to name resolution is the use of **aliases**
 - An alias is another name for the same entity
 - E.g., An environment variable is an example of an alias
- In terms of naming graphs, there are essentially two ways to implement aliases
 1. Via using *hard links*
 2. Via using *symbolic links*

Hard Links

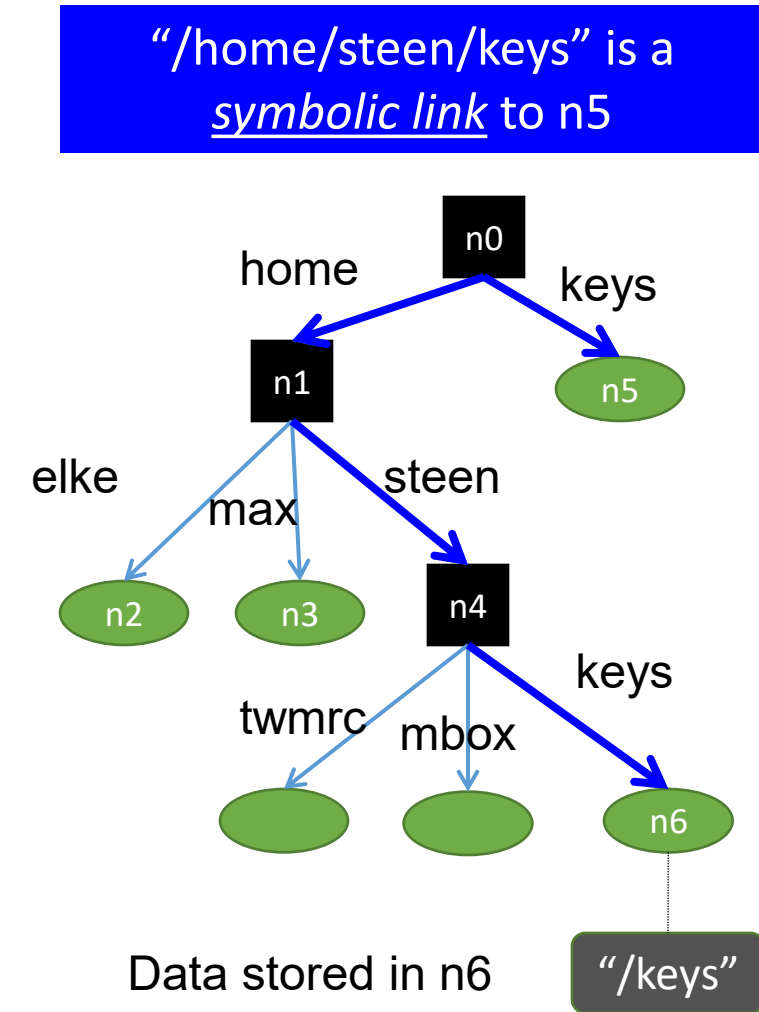
- Hard links allow multiple names to refer to the same node in the graph
- Name resolution:
 - Similar to the general name resolution
- Constraint:
 - There should be no cycles in the graph

“/home/steen/keys” and “/keys”
are hard links to n5



Symbolic Links

- Instead of storing in a leaf node an *address* or a *state*, symbolic links suggest storing an *absolute path name*
- Name resolution for a symbolic link, SL:
 - First resolve SL's name
 - Read the content of SL
 - Name resolution continues with the content of SL
- Constraint:
 - No cyclic references should be present



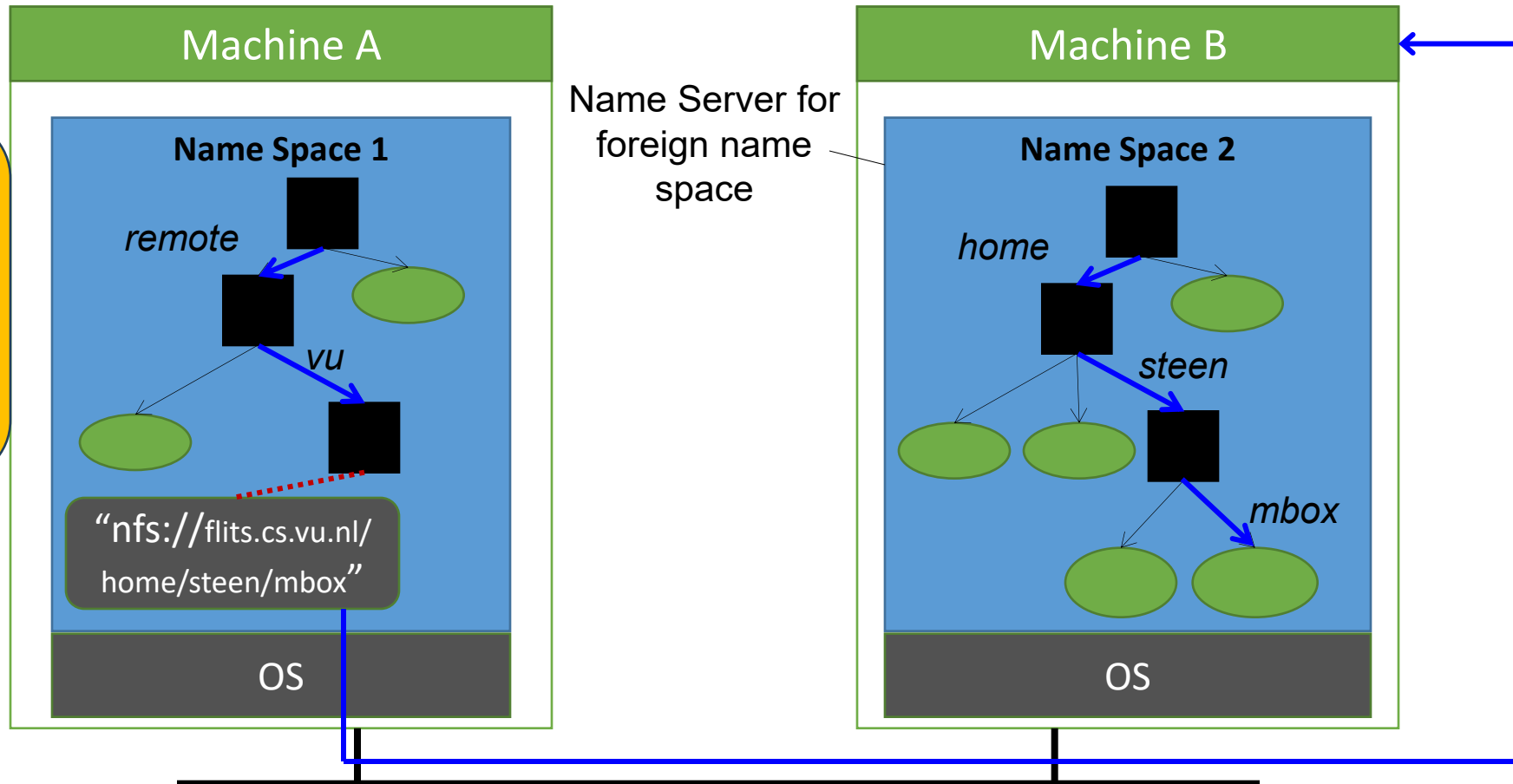
Mounting of Name Spaces

- Two or more name spaces can be merged transparently by a technique known as *mounting*
- With mounting, a directory node in one name space will store the identifier of a directory node of another name space
- The Network File System (NFS) is an example where different name spaces are mounted
 - NFS enables *transparent* access to remote files

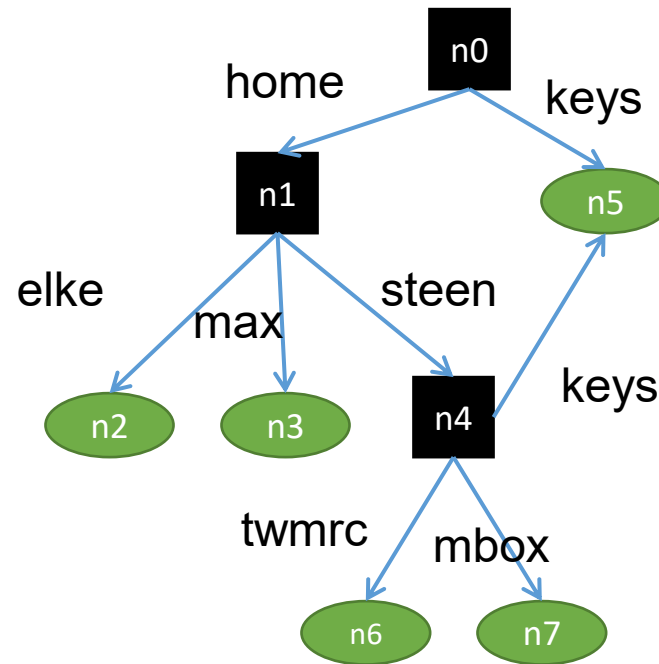
Example of Mounting Name Spaces in NFS

Lookup `"/remote/vu/home/steen/mbox"`

Knowing *from where* to start is referred to as a ***closure mechanism***

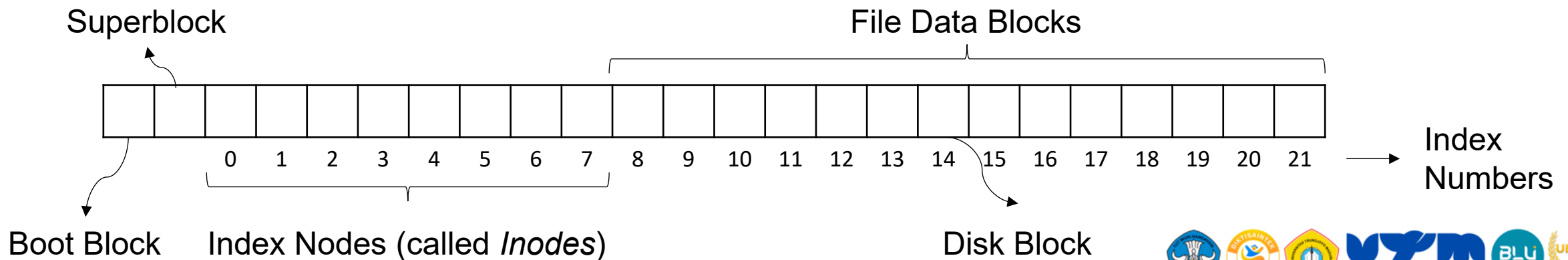


How to Distribute the Unix Naming Graph?



← Logical Representation

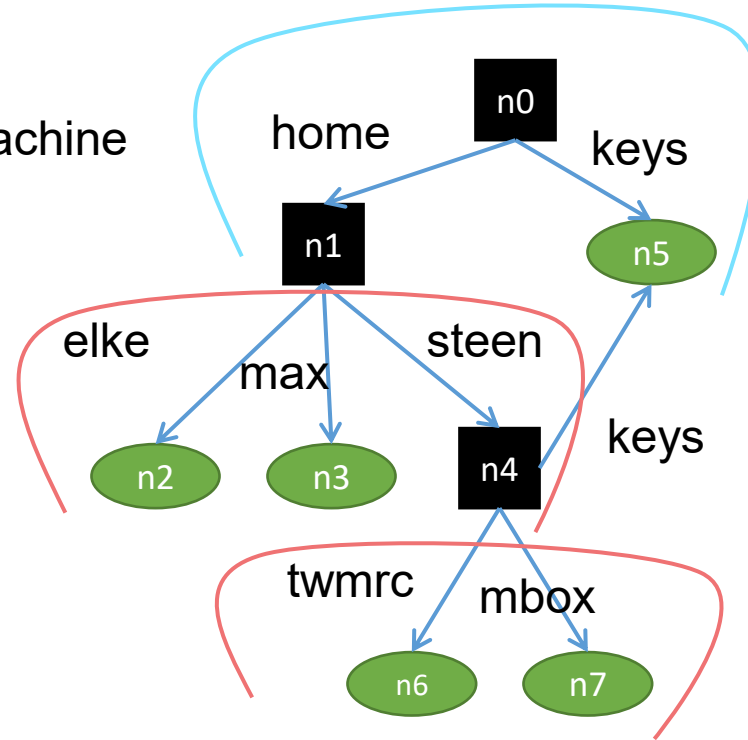
Physical Representation



How to Distribute the Unix Naming Graph?

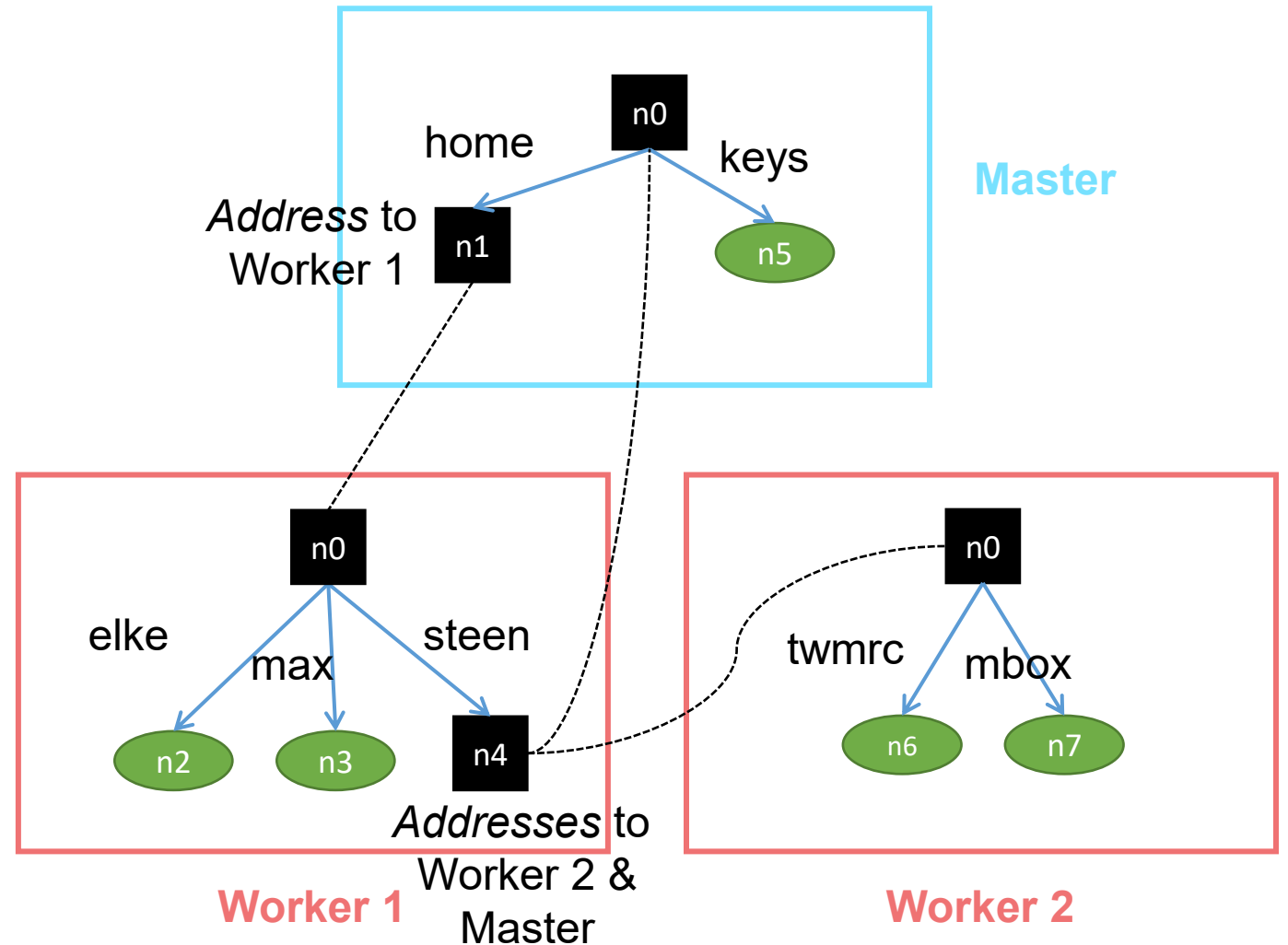
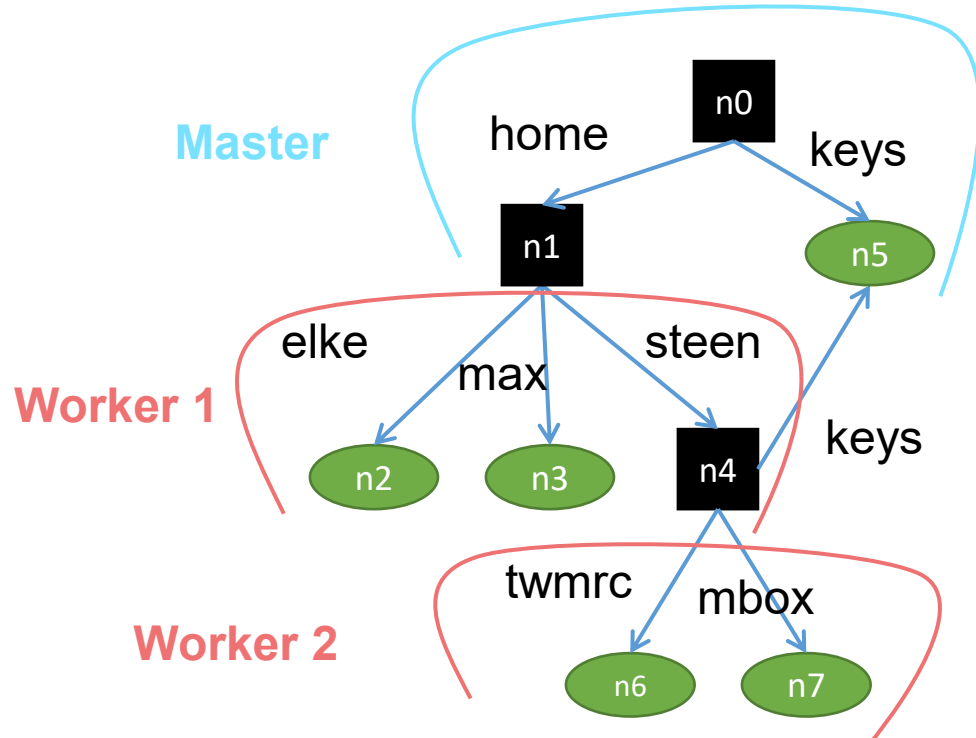
Map this to a *master* machine

Map this to a *worker* machine



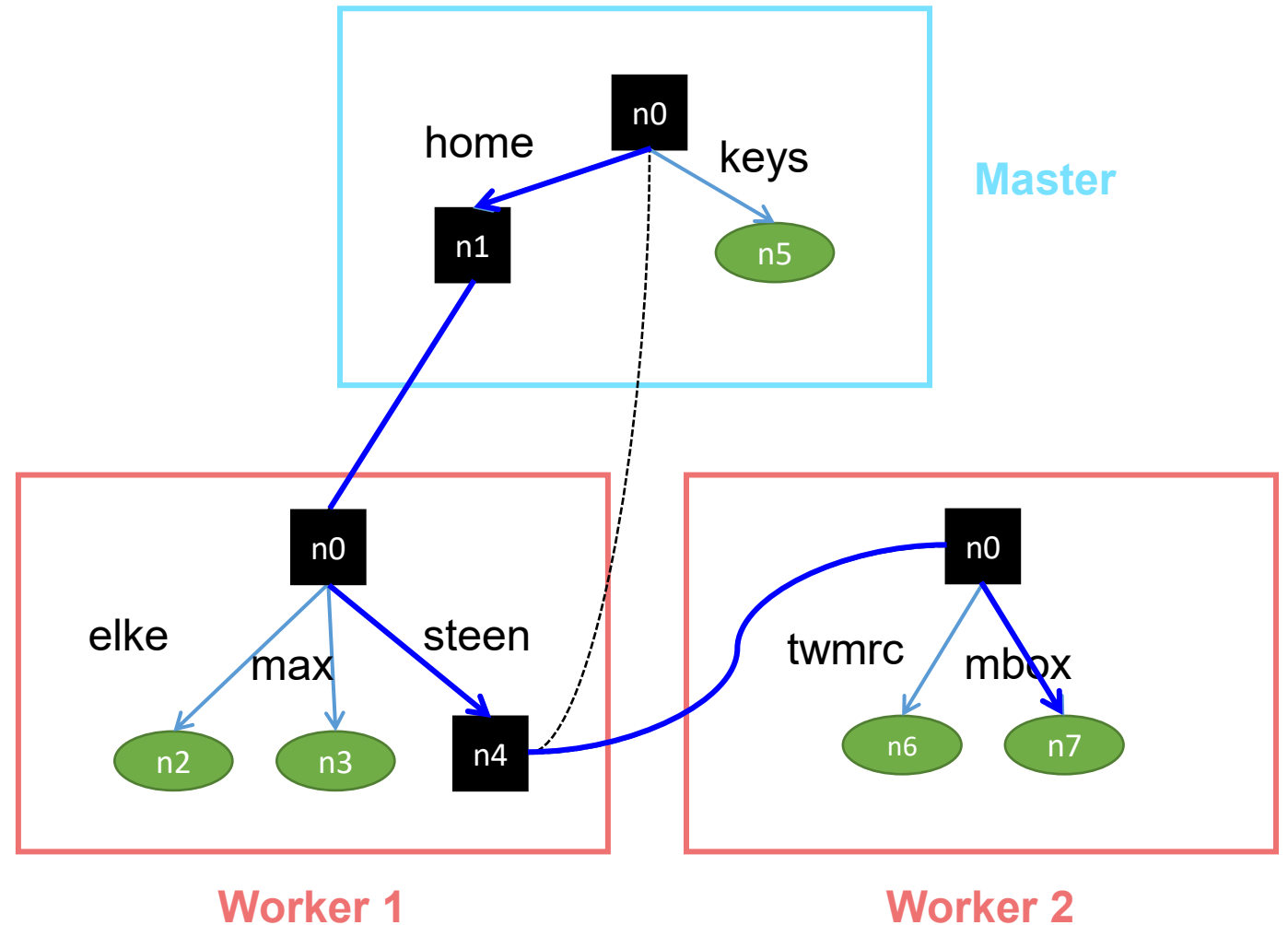
Map this to a *worker* machine

How to Distribute the Unix Naming Graph?



How to Distribute the Unix Naming Graph?

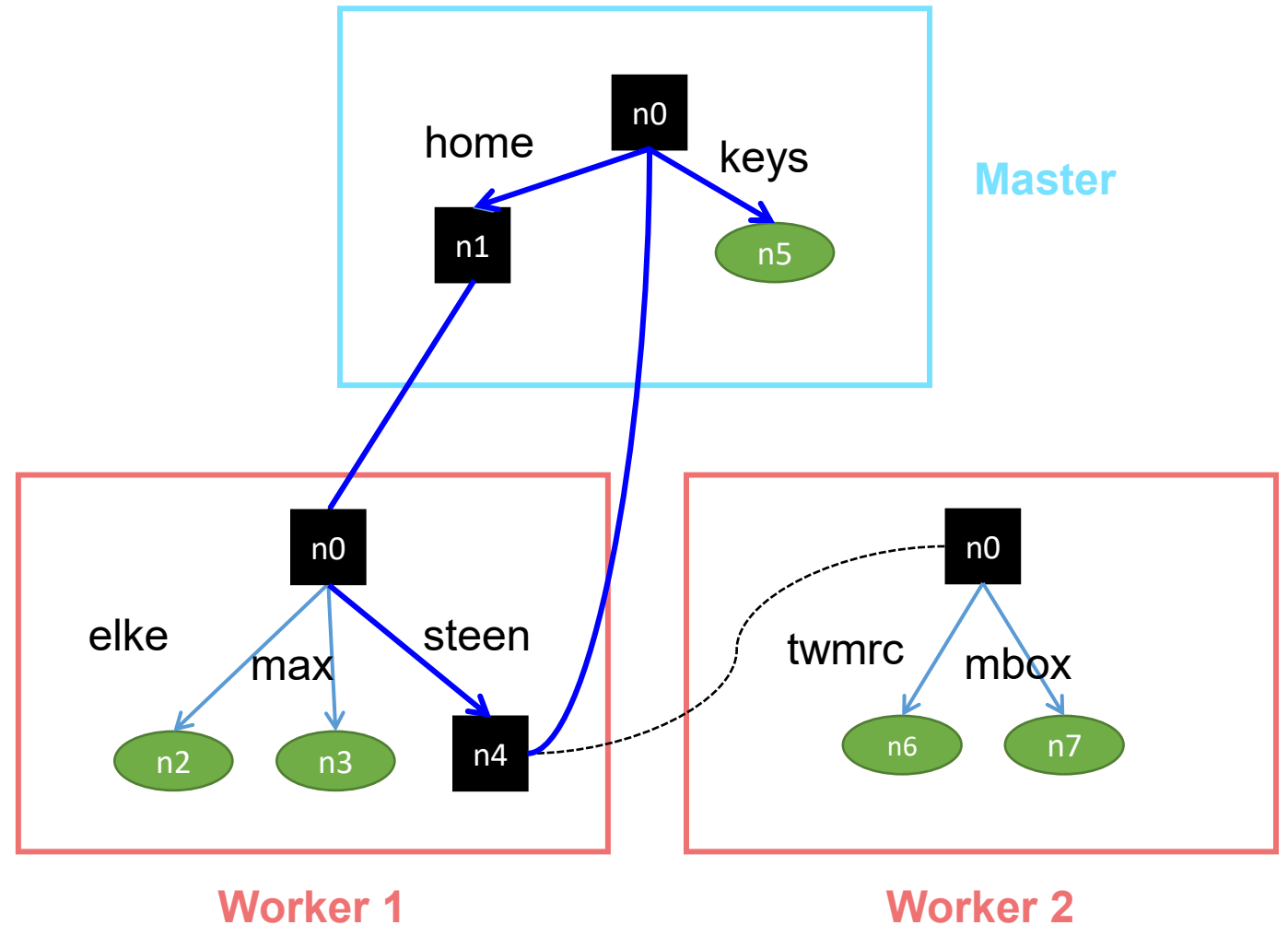
Lookup `"/home/steen/mbox"`



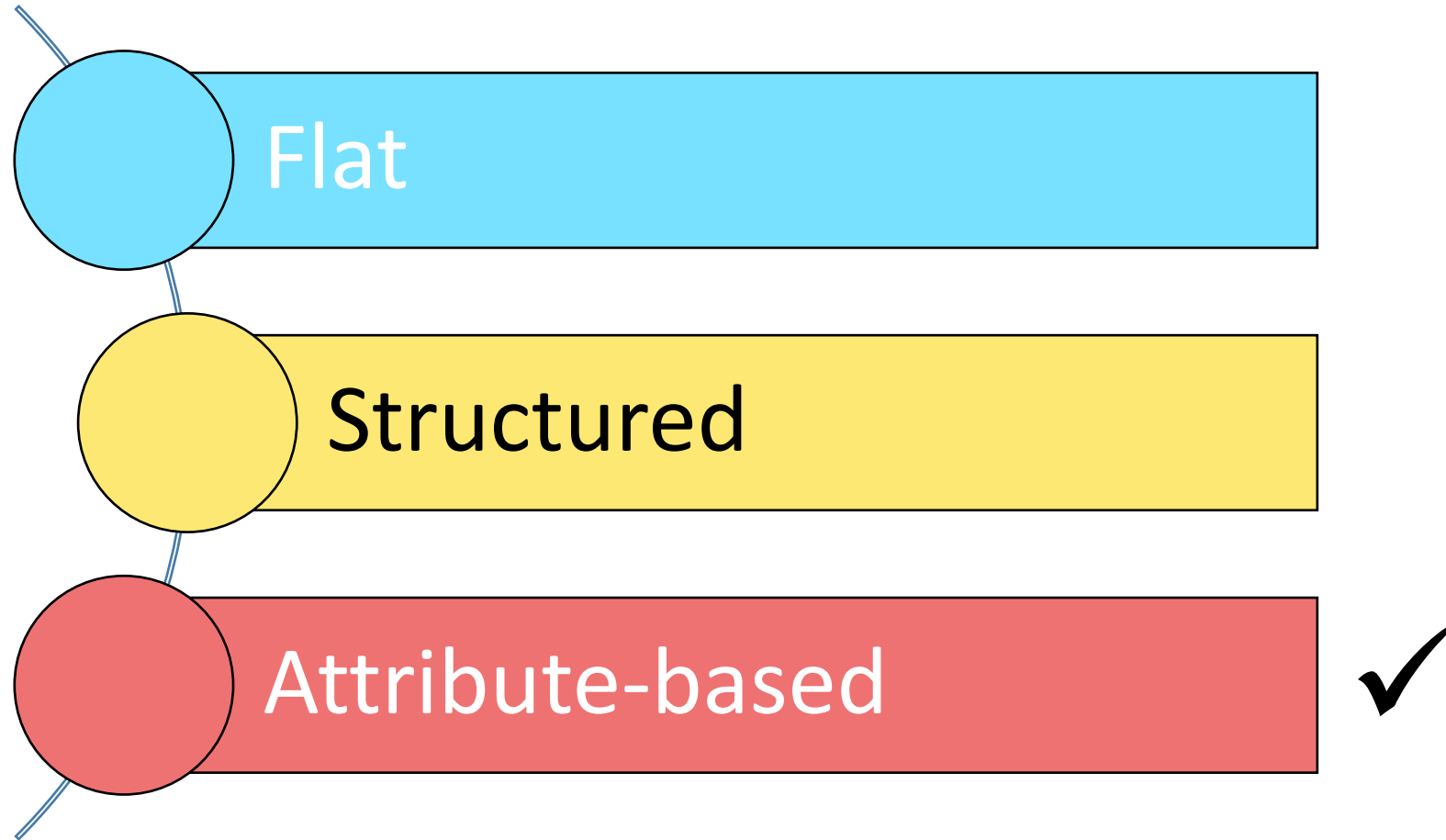
How to Distribute the Unix Naming Graph?

Lookup “/home/steen/keys”

You can also distribute the
Unix naming graph using a
peer-to-peer architecture
(*left as an exercise*)



Classes of Naming



Attribute-based Naming

- In many cases, it is much more convenient to name, and look up entities by means of their attributes
 - Similar to traditional directory services (e.g., yellow pages)
- However, the lookup operations can be extremely expensive
 - They require to match requested attribute values, against actual attribute values, which might require inspecting all entities
- Idea:
 - Implement basic directory service as a *database*, and combine it with traditional structured naming

Example: Lightweight Directory Access Protocol

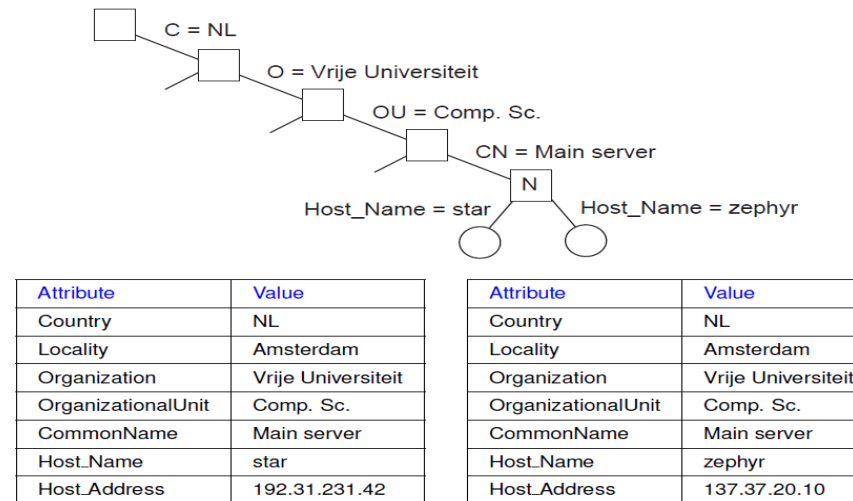
- **L**ightweight **D**irectory **A**ccess **P**rotocol (LDAP) directory service consists of a number of records called “directory entries”
 - Each record is made of (attribute, value) pairs
 - LDAP standard specifies five attributes for each record
- Directory Information Base (DIB) is a collection of all directory entries
 - Each record in a DIB is unique
 - Each record is represented by a distinguished name

E.g., /C=NL/O=Vrije Universiteit/OU=Comp. Sc.

Attribute	Value
Country	NL
Locality	Amsterdam
Organization	Vrije Universiteit
OrganizationalUnit	Comp. Sc.
CommonName	Main server
Host_Name	star
Host_Address	192.31.231.42

Directory Information Tree in LDAP

- All the records in the DIB can be organized into a hierarchical tree called the *Directory Information Tree (DIT)*



- LDAP provides advanced search mechanisms based on attributes by traversing the DIT
 - Example syntax for searching all Main_Servers in Vrije Universiteit:

```
search("&(C = NL) (O = Vrije Universiteit) (OU = *) (CN = Main server)")
```



Summary

- Naming allows locating entities in a distributed system
- Three types of naming:
 - Flat Naming
 - Broadcasting, forwarding pointers, home-based approach, Distributed Hash Table (DHT)
 - Structured Naming
 - Organizes names into name spaces (or *naming graphs*)
 - Distributes name spaces across machines
 - Attribute-based Naming
 - Entities are looked up using their attributes



Next Lecture...

- Synchronization – Part I